



UNIVERSITÉ DE SFAX

FACULTÉ DES SCIENCES ÉCONOMIQUES ET DE GESTION

DÉPARTEMENT D'INFORMATIQUE APPLIQUÉE À LA GESTION

Mémoire de Fin d'Études

**Pour l'obtention de la licence en informatique appliquée à la
gestion**

Plateforme de Financement Collaboratif

Réalisé par : Gdoura Karim

Encadré par : M. Iskandar Keskes

Jury

Président :

Membre :

Membre :

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer toute ma reconnaissance aux personnes qui ont contribué, directement ou indirectement, à la réalisation de ce travail.

J'adresse mes remerciements tout particuliers à mon encadreur académique, **M. Iskandar Keskes**.

Je remercie également les membres du jury qui acceptent d'examiner ce mémoire et d'en apprécier la teneur, ainsi que l'attention qu'ils y porteront.

Je suis reconnaissant envers l'équipe pédagogique du département d'informatique appliquée à la gestion pour la qualité de la formation assurée.

Enfin, j'associe à ce travail ma famille et mes proches : leur soutien a constitué tout au long de ce parcours un facteur durable de stimulation et d'engagement.

Table des matières

Remerciements	i
Liste des abréviations	ix
Introduction générale	1
1 Définition du projet et analyse des solutions existantes	4
1.1 Introduction	4
1.2 Définition de la mission	4
1.2.1 Présentation de l'organisme d'accueil	4
1.2.2 Présentation de l'application	5
1.2.3 Objectifs à atteindre	6
1.3 Contexte du sujet	6
1.4 Analyse d'applications existantes	7
1.4.1 Application A : plateformes internationales de référence	7
1.4.2 Application B : plateformes tunisiennes	8
1.5 Problématique et solution proposée	9
1.6 Conclusion	11
2 Capture des besoins	12
2.1 Introduction	12

2.2	Identification des acteurs du système informatisé	12
2.3	Contexte du système informatisé (boîte noire)	13
2.4	Identification des cas d'utilisation	14
2.5	Description textuelle des cas d'utilisation	16
2.5.1	Inventaire exhaustif des cas d'utilisation	16
2.5.2	Fiches détaillées (format annexe)	19
2.6	Conclusion	24
3	Analyse et conception	25
3.1	Introduction	25
3.2	Réalisation conceptuelle des cas d'utilisation	25
3.2.1	Annuler une contribution	27
3.2.2	Créer et soumettre un projet	29
3.2.3	Investir dans un projet (paiement démo + <i>webhook</i>)	31
3.2.4	Valider un projet (Admin)	33
3.2.5	Fournir les coordonnées bancaires	35
3.2.6	Approuver un retrait (Admin)	37
3.2.7	Mise à jour de l'état d'un projet (Admin)	39
3.3	Modèle du domaine	40
3.4	Architecture du futur logiciel	42
3.4.1	Vue d'ensemble en couches	42
3.4.2	Intégration de n8n	42
3.4.3	Composants asynchrones (BullMQ / Redis)	43
3.4.4	Vue logique en paquetages	43
3.4.5	Décisions de conception structurantes	44
3.5	Modèle de données MongoDB	45
3.5.1	Collections et structure des documents	45

3.5.2	Justification du modèle NoSQL	45
3.5.3	Indexation et relations	46
3.5.4	Dictionnaire des données	47
3.6	Conclusion	49
4	Implémentation et tests	51
4.1	Introduction	51
4.2	Environnement de réalisation	51
4.2.1	Environnement logiciel	51
4.2.2	Conteneurisation (Docker) et expérimentation n8n	57
4.2.3	Gestion de version et collaboration (GitHub)	58
4.2.4	Synthèse des choix technologiques	58
4.3	Réalisation technique des cas d'utilisation	59
4.3.1	Authentification et gestion des utilisateurs	59
4.3.2	Gestion des projets	63
4.3.3	Parcours de paiement (simulation)	66
4.3.4	Nettoyage des investissements abandonnés	69
4.3.5	Moteur de recommandation	70
4.3.6	Service de notifications	72
4.3.7	Stratégie de test	73
4.3.8	Déploiement	74
4.4	Conclusion	74
	Conclusion générale	76
	Webographie	79
	Annexe	82

Format type d'une description textuelle de cas d'utilisation	82
--	----

Table des figures

2.1	Vue de contexte du système (boîte noire) — diagramme de collaboration UML entre acteurs externes et le SI	14
2.2	Diagramme de cas d'utilisation	15
3.1	Diagramme de séquence – Annuler une contribution	27
3.2	Diagramme de classes participantes – Annuler une contribution	28
3.3	Diagramme de séquence – Créer et soumettre un projet	29
3.4	Diagramme de classes participantes – Créer et soumettre un projet	30
3.5	Diagramme de séquence – Investir (paiement démo + <i>webhook</i>)	31
3.6	Diagramme de classes participantes – Investir dans un projet (paiement démo + <i>webhook</i>)	32
3.7	Diagramme de séquence – Valider un projet	33
3.8	Diagramme de classes participantes – Valider ou rejeter un projet (Admin) . . .	34
3.9	Diagramme de séquence – Fournir les coordonnées bancaires	35
3.10	Diagramme de classes participantes – Fournir les coordonnées bancaires	36
3.11	Diagramme de séquence – Approuver un retrait (Admin)	37
3.12	Diagramme de classes participantes – Approuver un retrait (Admin)	38
3.13	Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)	39
3.14	Diagramme de classes participantes – Suspendre ou réactiver un projet (Admin)	40
3.15	Diagramme de classes du domaine (modèle du domaine)	41
3.16	Diagramme de paquetages de l'application (<code>diagramme_de_paquetage/diagramme_paquetage.p</code>	

4.1	Page de connexion (screens/login.png)	61
4.2	Formulaire d'inscription (screens/inscription.png)	61
4.3	Vérification d'e-mail : saisie du code (screens/codeverifemail.png) . . .	62
4.4	Page de vérification / confirmation après validation (screens/pageverif.png)	62
4.5	Workflow n8n — analyse de risque (interface d'édition); fichier screens/n8nriskanalysis.png	63
4.6	Orchestration n8n complémentaire / scénarios planifiés; fichier screens/n8ncrons.png	64
4.7	Formulaire de création de projet — étape 1 (screens/creationdeprojet1.png)	65
4.8	Création de projet — étape complémentaire (screens/creationdeprojet2.png)	65
4.9	Console d'administration : validation/rejet (screens/validerrejeter.png)	66
4.10	Formulaire de soutien : investir (screens/investir.png)	67
4.11	Étape de confirmation (code OTP / sécurisation du paiement démo) (screens/codeOTPinvestir.png)	
4.12	Coordonnées bancaires (screens/cordonnebancaire.png); chiffrement AES-256-GCM en amont stockage	68
4.13	Contributions / investissements (screens/mescontributions.png); statuts INITIATED, SUCCESS, FAILED	70
4.14	Recommandations de projets (screens/recommendation.png)	71
4.15	Notifications (screens/notification.png) : liste personnelle paginée . . .	73

Liste des tableaux

1.1	Comparaison des plateformes de crowdfunding	10
2.1	Inventaire des cas d'utilisation — paquetage Comptes	17
2.2	Inventaire des cas d'utilisation — paquetage Projets	17
2.3	Inventaire des cas d'utilisation — paquetage Contributions & finance	18
2.4	Inventaire des cas d'utilisation — paquetage Échanges & confiance	18
2.5	Inventaire des cas d'utilisation — paquetage Administration	19
2.6	Description textuelle du cas « Créer et soumettre un projet »	20
2.7	Description textuelle du cas « Soutenir un projet »	21
2.8	Description textuelle du cas « Valider / rejeter un projet »	22
2.9	Description textuelle du cas « Voir les projets recommandés »	23
2.10	Description textuelle du cas « Signaler un contenu »	23
3.1	Dictionnaire des données	47
4.1	Technologies utilisées — socle applicatif, données et sécurité	58
4.2	Technologies utilisées — IA, orchestration et exploitation (suite)	59
4.3	Modèle de description textuelle d'un cas d'utilisation	82

Liste des abréviations

Abrév.	Signification
AES-GCM	Advanced Encryption Standard - Galois/Counter Mode (chiffrement authentifié)
API	Application Programming Interface (interface de programmation)
BCT	Banque Centrale de Tunisie
BullMQ	File de jobs Node.js bâtie sur Redis
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
DLQ	Dead-Letter Queue (file des échecs définitifs)
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
IA	Intelligence Artificielle
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model (modèle de langage de grande taille)
MVC	Modèle-Vue-Contrôleur
n8n	Outil d'orchestration de workflows
ORM	Object-Relational Mapping (mapping objet-relationnel)
OTP	One-Time Password (code à usage unique)
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control (contrôle d'accès par rôle)
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
RIB	Relevé d'Identité Bancaire
SPA	Single Page Application
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TND	Dinar tunisien
UML	Unified Modeling Language

Introduction générale

Contexte général

Le financement participatif (*crowdfunding*) constitue une voie complémentaire aux dispositifs de financement classiques. En Tunisie, son développement s'est nettement accentué depuis l'adoption de la loi n° 2020-37 du 2 juillet 2020, laquelle encadre les opérations concernées et fixe les principes de supervision relevant de la Banque Centrale de Tunisie (BCT).

Ce mouvement s'inscrit dans une dynamique élargie de mutation numérique de l'économie tunisienne, au sein de laquelle start-ups et PME rencontrent fréquemment des obstacles d'accès au crédit bancaire traditionnel. Dans ce cadre, le financement collaboratif permet de mobiliser l'épargne au service de projets entrepreneuriaux tout en ouvrant aux investisseurs des possibilités complémentaires de allocation de capitaux.

Problématique

L'examen des plateformes de financement collaboratif existantes conduit à souligner plusieurs limites récurrentes :

- Absence d'évaluation automatisée des risques pour guider les investisseurs dans leurs choix
- Manque de personnalisation des recommandations de projets selon les profils des investisseurs
- Processus de validation manuelle des projets chronophage pour les administrateurs
- Intégration paiement souvent complexe (webhooks, idempotence, sécurité) et difficile à tester sans sandbox

Face à ces constats, la question de recherche suivante se pose : **Comment concevoir une plateforme de financement collaboratif intégrant l'intelligence artificielle pour automatiser**

l'évaluation des risques et personnaliser les recommandations, tout en garantissant la conformité réglementaire et la sécurité des transactions ?

Objectifs du projet

Le présent travail a pour ambition de concevoir et de réaliser une plateforme web de financement participatif répondant aux objectifs ci-après énumérés :

Objectifs fonctionnels :

1. Permettre aux porteurs de projets de créer, soumettre et gérer leurs campagnes de financement
2. Offrir aux investisseurs une interface intuitive pour découvrir, évaluer et financer des projets
3. Automatiser le processus de validation des projets via une analyse de risque basée sur l'IA
4. Personnaliser les recommandations de projets selon le profil et les préférences de chaque investisseur
5. Proposer un parcours de paiement démonstratif (mock) et une architecture extensible vers un prestataire réel
6. Fournir un tableau de bord administrateur pour superviser les opérations et gérer les litiges

Objectifs techniques :

1. Concevoir une architecture modulaire et scalable (Node.js, React, MongoDB)
2. Implémenter un moteur de recommandation déterministe (profil investisseur, catégories, niveau de risque issu de l'analyse IA des projets, popularité et récence)
3. Intégrer un service LLM pour l'analyse de risque (rapport structuré) via un workflow n8n
4. Orchestrer les workflows asynchrones via n8n (analyse IA) et des files BullMQ (e-mails, reprises)
5. Garantir la résilience du système avec gestion des échecs (dead-letter queues)
6. Assurer la sécurité des données (chiffrement, JWT, HTTPS)

Organisation du mémoire

Le document est structuré en quatre chapitres, conformément au *Plan du rapport PFE — version 2*. Le premier chapitre établit le cadre général : mission, contexte du sujet, analyse des solutions existantes, problématique et solution proposée. Le second chapitre formalise la capture des besoins (acteurs, contexte « boîte noire », identification des cas d'utilisation, inventaire exhaustif aligné sur le code **livré**, puis fiches détaillées au format annexe pour des cas transverses majeurs). Le troisième chapitre traite de l'analyse et de la conception (réalisation conceptuelle des cas d'utilisation, modèle du domaine, architecture du futur logiciel et **modèle de données MongoDB**). Le quatrième chapitre expose l'implémentation et les tests (environnement de réalisation, réalisation technique des cas d'utilisation, stratégie de validation et déploiement). La conclusion générale synthétise les résultats, identifie les apports, les limites et les perspectives d'évolution. Une annexe rappelle le modèle de fiche employé pour les descriptions textuelles de cas d'utilisation (chapitre 2).

Chapitre 1

Définition du projet et analyse des solutions existantes

1.1 Introduction

Le présent chapitre dresse le cadre général du projet. Il expose la mission définie (organisme d'accueil, application visée, objectifs assignés), contextualise le sujet sur les plans économique et réglementaire, examine les principales solutions concurrentes sur le segment du financement participatif, puis formule la problématique et la solution retenue. Il constitue en ce sens le socle interprétatif des besoins développés au chapitre suivant.

1.2 Définition de la mission

1.2.1 Présentation de l'organisme d'accueil

Le présent projet de fin d'études relève d'un stage académique placé sous la responsabilité de M. Iskandar Keskes, enseignant à la Faculté des Sciences Économiques et de Gestion (FSEG) de Sfax, au sein du département d'Informatique Appliquée à la Gestion. L'ensemble du travail a été conduit en autonomie, tout en étant jalonné de réunions de suivi hebdomadaires destinées à valider les décisions de conception, à préciser les priorités et à trancher les arbitrages fonctionnels et techniques attendus dans un tel dispositif. Sur le plan méthodologique, l'exercice s'est inscrit explicitement dans une perspective d'*ingénierie des systèmes d'information*, associant capture des

besoins, modélisation UML, conception des données puis réalisation et validation au moyen de scénarios métier, en cohérence avec les objectifs du cursus de licence en Informatique Appliquée à la Gestion.

1.2.2 Présentation de l'application

L'application réalisée est une **plateforme web de financement collaboratif** (*crowdfunding*) qui met en relation trois catégories d'acteurs métier :

- les **porteurs de projet** qui souhaitent financer une campagne (titre, description, catégorie, objectif financier, échéance, médias) ;
- les **investisseurs** qui souhaitent contribuer à des campagnes correspondant à leur profil (catégories préférées, tolérance au risque) ;
- les **administrateurs** qui assurent la gouvernance de la plateforme (validation des projets, modération des contenus, traitement des signalements, supervision des retraits).

Les concepts métier centraux portés par l'application sont les suivants :

- **Projet** : campagne de financement caractérisée par un cycle de vie explicite (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED).
- **Investissement** : engagement d'un investisseur pour un montant donné sur un projet, créé en INITIATED puis confirmé en SUCCESS (ou REFUNDED en cas de sur-financement) après réception du *webhook* de paiement.
- **Analyse de risque** : rapport structuré (résumé, avantages, inconvénients, améliorations, éléments à retirer, questions, score/riskLevel) produit par un service LLM orchestré par n8n, et destiné à éclairer la revue administrative.
- **Recommandations personnalisées** : classement des projets accessibles à un investisseur en fonction de ses catégories préférées et de sa tolérance au risque.
- **Notifications** : chaque message *in-app* est stocké en base avec un destinataire explicite (userId) ; l'interface expose une boîte **personnelle** identique pour tous les rôles (y compris l'administrateur), avec liste paginée (30 entrées par page, ordre antéchronologique sur createdAt), marquage lu/non lu et compteur global de non-lus pour le badge. Les e-mails asynchrones ne concernent que les événements jugés critiques (politique métier), afin de limiter le bruit.
- **Retrait (payout)** : flux de versement au porteur de projet une fois la campagne FUNDED ou CLOSED, après saisie chiffrée des coordonnées bancaires et approbation administrative.

1.2.3 Objectifs à atteindre

La plateforme doit répondre aux objectifs suivants, formulés à la fois sur le plan métier et sur le plan technique.

Objectifs métier :

- Permettre aux porteurs de projets de créer, soumettre, publier et suivre leurs campagnes de financement.
- Offrir aux investisseurs une expérience claire de découverte, d'évaluation et de contribution aux projets.
- Automatiser l'évaluation préliminaire du risque pour fiabiliser et accélérer la revue administrative.
- Personnaliser les recommandations selon le profil de l'investisseur (catégories, tolérance au risque).
- Assurer la traçabilité des transactions (statuts d'investissement, de transaction et de *payout*).
- Outiller la gouvernance de la plateforme (validation, modération, signalements, retraits, statistiques).

Objectifs techniques :

- Concevoir une architecture modulaire (Node.js / Express + React + MongoDB) avec une séparation nette entre logique métier et accès aux données.
- Mettre en place un parcours de paiement de démonstration (*mock*) avec confirmation par *webhook* signé et idempotent, extensible vers un prestataire réel.
- Orchestrer les traitements asynchrones (analyse IA, notifications, reprises) via n8n et des files BullMQ/Redis, avec stockage des échecs définitifs (*dead-letter queues*).
- Garantir la sécurité (authentification par JWT en cookies HttpOnly, contrôle d'accès par rôle, *rate limiting*, signature des *webhooks*, chiffrement AES-256-GCM des coordonnées bancaires).

1.3 Contexte du sujet

Le financement participatif (*crowdfunding*) constitue une alternative innovante aux modes de financement bancaires traditionnels. En Tunisie, le secteur connaît un essor important depuis la promulgation de la **loi n° 2020-37 du 2 juillet 2020**, qui encadre les opérations de financement participatif et place leur supervision sous la tutelle de la Banque Centrale de Tunisie (BCT). Ce

cadre réglementaire ouvre la voie à des plateformes qui mobilisent l'épargne publique au profit de projets entrepreneuriaux, tout en imposant des exigences fortes en matière d'identification des acteurs, de traçabilité des opérations et de conservation des journaux financiers.

Sur le plan économique, ce mouvement répond à une difficulté concrète des start-ups et PME tunisiennes pour accéder aux financements bancaires classiques, en particulier pour les projets innovants ou portant un risque élevé. Sur le plan technologique, l'émergence des modèles de langage (LLM) et des outils d'orchestration de *workflows* (par exemple n8n) permet d'automatiser une partie de la pré-évaluation des projets et de fiabiliser les flux asynchrones (paiements, notifications, reprises) sans alourdir la base de code applicative. Le sujet du présent projet s'inscrit donc à l'intersection de la **conformité réglementaire**, de l'**ingénierie des systèmes d'information** et de l'**IA appliquée** à la décision métier.

1.4 Analyse d'applications existantes

Cette section présente une étude comparative des principales plateformes de financement participatif, en distinguant les acteurs internationaux qui ont structuré le marché et les acteurs locaux qui en illustrent la déclinaison tunisienne. L'objectif est d'identifier les fonctionnalités déjà offertes et, surtout, les manques que la solution proposée vise à combler.

1.4.1 Application A : plateformes internationales de référence

Kickstarter

Kickstarter est la plateforme pionnière du crowdfunding, lancée en 2009. Elle se concentre sur le financement de projets créatifs (arts, design, technologie). Le modèle "tout ou rien" (all-or-nothing) impose que le projet atteigne 100% de son objectif pour que les fonds soient débloqués.

Points forts :

- Grande notoriété et trafic important
- Communauté engagée de créateurs
- Processus de validation qualité rigoureux

Points faibles :

- Absence d'analyse de risque automatisée
- Pas de personnalisation des recommandations

- Frais élevés (5% + 3-5% frais de paiement)

Indiegogo

Indiegogo offre plus de flexibilité que Kickstarter avec deux modèles : “tout ou rien” et “flexible funding” (les fonds sont conservés même si l’objectif n’est pas atteint).

Points forts :

- Flexibilité du modèle de financement
- Support de campagnes en continu (pas de deadline obligatoire)
- Partenariats avec des fabricants (InDemand)

Points faibles :

- Taux de succès des campagnes souvent plus faible que les leaders du marché (données publiées hétérogènes)
- Interface moins intuitive
- Manque de filtrage intelligent

1.4.2 Application B : plateformes tunisiennes

CoFundy

CoFundy est une plateforme tunisienne de crowdfunding lancée en 2016, spécialisée dans le financement de projets entrepreneuriaux.

Caractéristiques :

- Intégration avec paiements tunisiens (cartes bancaires locales)
- Validation manuelle des projets (délai 7-10 jours)
- Frais : 5% du montant collecté

Limitations identifiées :

- Absence de recommandations personnalisées
- Interface utilisateur datée
- Pas d’évaluation de risque visible pour les investisseurs
- Intégration paiement souvent limitée / difficile à tester sans sandbox

Kaoun

Plateforme tunisienne plus récente (2019), focalisée sur les projets sociaux et environnementaux.

Points forts :

- Niche bien définie (impact social)
- Communauté engagée

Points faibles :

- Volume de projets limité
- Fonctionnalités basiques
- Pas d'IA ou d'automatisation

1.5 Problématique et solution proposée

La revue des plateformes existantes met en évidence quatre manques structurels qui pénalisent à la fois les investisseurs, les porteurs de projet et les administrateurs.

Manques observés :

1. Absence d'évaluation automatisée des risques pour guider les investisseurs et accélérer la revue.
2. Manque de personnalisation des recommandations selon le profil de l'investisseur (catégories, tolérance au risque).
3. Processus de validation des projets manuels et chronophages pour les administrateurs.
4. Intégration paiement souvent peu transparente, difficile à tester sans environnement de bac à sable, et faiblement résiliente face aux échecs (annulations, remboursements, sur-financement).

Le tableau 1.1 synthétise ces écarts et positionne la solution proposée par rapport aux plateformes étudiées.

Critère	Kickstarter	Indiegogo	CoFundy	Kaoun	Notre solution
Analyse IA	Non	Non	Non	Non	Oui
Recommandations	Non	Basique	Non	Non	Personnalisées
Paiement TN	Non	Non	Limité	Oui	Démo (<i>mock</i>) / extensible
Gestion des échecs	Manuelle	Manuelle	Manuelle	Manuelle	Automatisée (<i>retries</i> + DLQ)
Frais	8–10%	8–13%	5%	5%	— (hors périmètre du prototype PFE)

TABLE 1.1 – Comparaison des plateformes de crowdfunding

Question de recherche. Comment concevoir une plateforme de financement collaboratif intégrant l’intelligence artificielle pour automatiser l’évaluation des risques et personnaliser les recommandations, tout en garantissant la conformité réglementaire (BCT) et la sécurité des transactions ?

Solution proposée. La plateforme développée combine quatre choix structurants :

- Une **analyse de risque automatisée** (LLM orchestré par n8n) qui produit un rapport structuré (résumé, avantages, inconvénients, améliorations, éléments à retirer, questions, score) lors de la soumission d’un projet, avec **auto-rejet** lorsque une heuristique de **cohérence budgétaire** (objectif fundingGoal vs estimation des besoins extraits de la description) détecte un **écart absolu** $\geq 30\%$ (règle bloquante), et relances automatiques en cas d’indisponibilité du workflow.
- Un **moteur de recommandations** entièrement déterministe dans le code livré (catégories préférées, adéquation au riskLevel issu de l’analyse IA du projet, progression et récence de la campagne), extensible ultérieurement par enrichissement LLM.
- Un **parcours de paiement de démonstration** (*mock*) avec confirmation par *webhook* signé et idempotent, conçu pour être remplacé par un prestataire réel sans changer la logique métier.
- Une **gestion résiliente des échecs** via files BullMQ/Redis et workflows n8n, avec stockage des événements définitivement échoués (*dead-letter queues*) pour reprise manuelle par l’administrateur.

1.6 Conclusion

Ce chapitre a permis de préciser la mission (organisme d'accueil, application visée, objectifs métier et techniques), de replacer le sujet dans son environnement économique et réglementaire, d'analyser les plateformes de référence retenues et d'en extraire la problématique centrale. La solution défendue repose sur une analyse par IA, sur des mécanismes de recommandation personnalisés, sur un parcours de paiement de démonstration extensible et sur une orchestration résiliente des traitements. Le chapitre suivant formalise les besoins métier à travers la description des acteurs et des cas d'utilisation.

Chapitre 2

Capture des besoins

2.1 Introduction

Ce chapitre formalise les besoins métier associés à la plateforme, du point de vue des acteurs qui en sont les utilisateurs ou les partenaires techniques. Il explicite le périmètre du système (représentation en « boîte noire »), recense les acteurs et leurs responsabilités respectives, structure les cas d'utilisation en paquetages métier cohérents, établit un **inventaire exhaustif** de ces cas (tableaux 2.1 à 2.5), puis illustre, pour un sous-ensemble **structurant**, des **descriptions détaillées au format annexe** (scénario nominal, variantes, exceptions) mises en correspondance avec les modules `/api/*` et `/internal/*` du backend livré.

2.2 Identification des acteurs du système informatisé

La plateforme distingue des acteurs humains et des acteurs systèmes externes. Un même utilisateur peut cumuler les rôles d'*investisseur* et de *porteur de projet* ; la distinction de ces deux rôles est portée dans la description des cas d'utilisation.

- **Visiteur** : utilisateur non authentifié ; consulte et recherche les projets publics, peut créer un compte et lancer une procédure de récupération de mot de passe.
- **Utilisateur authentifié** (porteur de projet et / ou investisseur) : gère son compte, crée et soumet des projets, contribue à des campagnes, commente et signale des contenus, suit ses investissements, ses paiements et ses retraits.
- **Administrateur** : valide, publie, suspend ou rejette les projets, modère les commentaires, traite les signalements, supervise les retraits, désactive ou réactive les comptes, et relance

les traitements échoués (notifications, retraits, remboursements).

- **Passerelle de paiement (d  mo)** : partenaire externe simul   qui confirme ou infirme les paiements via un *webhook* sign  . Les messages de ce tiers d  clenchent des transitions m  tier (statut de transaction, statut d'investissement, mise    jour du montant collect  , remboursements en cas de sur-financement).
- **Service d'analyse de risque (orchestrateur)** : service externe qui ex  cute les workflows d'analyse (sollicitation, reprises, mise    jour du rapport) et retourne un rapport structur   via les routes internes du backend (`/internal/*`); l'assistance conversationnelle sur un projet (`POST /api/projects/:id/chat`) est, quant    elle, impl  ment  e **dans l'API Express** avec appel direct    un service LLM c  t   serveur (sans passage par n8n).
- **Service de messagerie (SMTP)** : partenaire externe utilis   pour les e-mails critiques (v  rification, r  initialisation, d  cisions admin,   v  nements de paiement / retrait).

2.3 Contexte du syst  me informatis   (bo  te noire)

Du point de vue externe, la plateforme   change des messages avec **quatre familles d'interlocuteurs** (canal web humain, passerelle de paiement, service d'analyse de risque, messagerie). La figure 2.1 en restitue la lecture : le syst  me appara  t comme une « bo  te noire » centrale, les acteurs p  riph  riques   tant reli  s par des associations libell  es qui mat  rialisent les   changes fonctionnels attendus.

- Les **acteurs humains web** (visiteur, utilisateur authentifi  , administrateur) interagissent avec le syst  me via l'application front-end (HTTPS, REST, cookies HttpOnly).
- La **passerelle de paiement (d  mo)** envoie des confirmations sign  es (*webhook*); le syst  me r  pond de fa  on idempotente pour absorber les retentatives.
- Le **service d'analyse de risque (orchestrateur)** consomme des demandes d'analyse (via file de jobs/Redis) et appelle des routes internes du backend (`/internal/*`) pour mettre    jour les r  sultats (succ  s/  chec) de mani  re contr  l  e.
- Le **service de messagerie (SMTP)** est sollicit   pour les e-mails critiques (v  rification, r  initialisation, d  cisions admin, paiement/retrait).

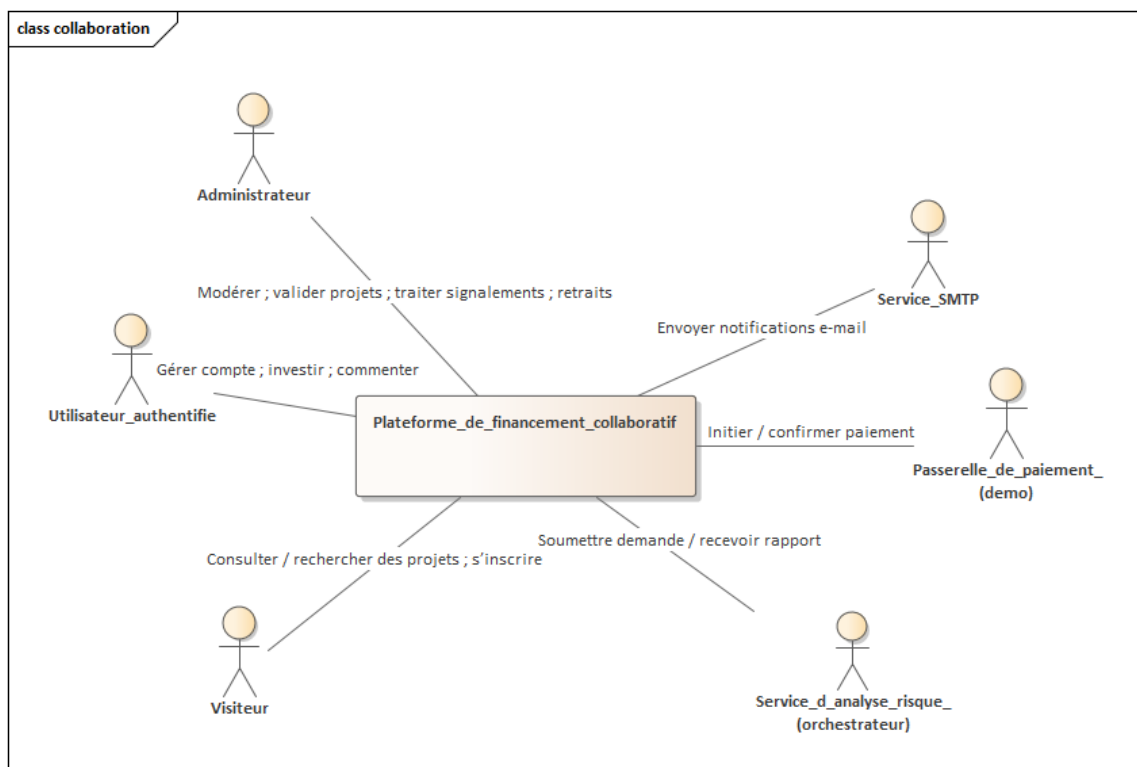


FIGURE 2.1 – Vue de contexte du système (boîte noire) — diagramme de collaboration UML entre acteurs externes et le SI

Le contour du logiciel se déduit de cette lecture : hors du noyau applicatif (persistance métier, API et interface utilisateur propre à la plateforme), chaque composant relève d'un partenaire externe et s'appuie sur un contrat d'interface explicite (REST, *webhook* signé, appels internes documentés).

2.4 Identification des cas d'utilisation

Les cas d'utilisation sont regroupés en thèmes métier qui structurent la plateforme (présentation synthétique). Les détails d'enchaînement et les variantes sont explicités dans les descriptions textuelles (section 2.5) et dans les diagrammes du chapitre 3.

- **Comptes** : inscription (avec OTP / lien de vérification), connexion, déconnexion, rafraîchissement de session, mot de passe oublié, mise à jour du profil, suppression de compte.
- **Projets** : exploration / consultation, recherche, création (DRAFT), soumission à l'analyse IA (AWAITING_AI), revue administrative (UNDER_REVIEW), décision (APPROVED/REJECTED),

- publication (ACTIVE), édition / re-soumission, archivage.
- **Contributions & finance** : contribution (paiement simulé), suivi des investissements, relance de paiement, annulation dans la fenêtre de grâce, remboursement en cas de sur-financement, retraits (*payouts*) et saisie chiffrée des coordonnées bancaires.
- **Échanges & confiance** : commentaires sur les projets publics (publication/suppression), notifications *in-app*, signalement (projet ou commentaire), consultation de ses signalements, recommandations personnalisées, chatbot d'aide à la consultation d'un projet.
- **Administration** : gestion des projets (liste, validation/rejet, publication, suspension/réactivation, relance analyse IA), gestion des utilisateurs (désactivation/réactivation), modération des commentaires (masquer/rétablir), traitement des signalements (résolution), supervision des retraits (approbation), relance des jobs en échec (envois SMTP, payouts, remboursements, annulations; files DLQ / routes /api/admin/*).

La représentation graphique synthétique de ces cas d'utilisation est donnée par la figure 2.2.

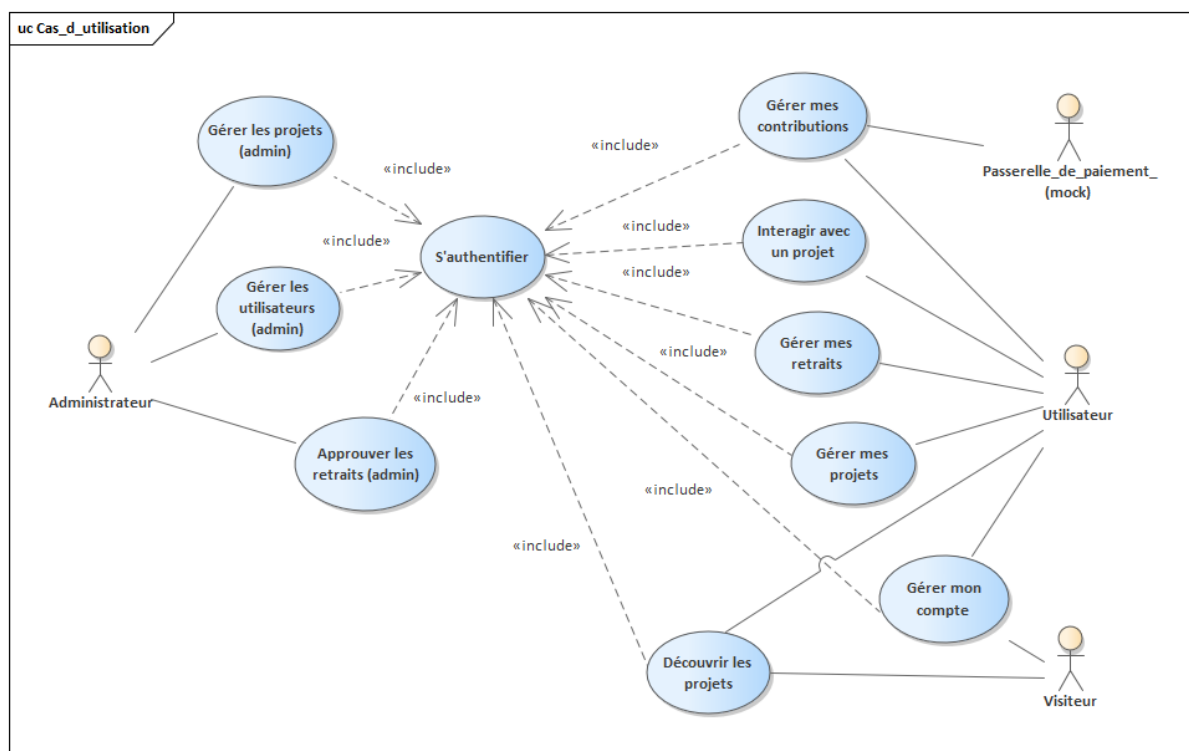


FIGURE 2.2 – Diagramme de cas d'utilisation

Lecture du diagramme. Le diagramme positionne les acteurs autour du système et regroupe les cas d'utilisation par thèmes métier. Les liens «include» (lorsqu'ils sont utilisés) permettent

de factoriser des sous-fonctionnalités obligatoires et réutilisées (par exemple l’authentification), tandis que les variantes sont détaillées dans les scénarios textuels (section 2.5) et dans les diagrammes d’interaction du chapitre 3.

Alignement avec l’implémentation. La couverture fonctionnelle du diagramme correspond aux modules exposés par le backend (/api/auth, /api/users, /api/projects, /api/investments, /api/webhooks, /api/payouts, /api/notifications, /api/reports, /api/recommendations, /api/projects/:id/chat (chatbot), /api/admin, /internal/*). Les tableaux 2.1 à 2.5 en donnent la cartographie exhaustive ; le chapitre 3 en reprend une partie sous forme UML.

2.5 Description textuelle des cas d’utilisation

Conformément au plan du rapport (version 2), la capture des besoins exige à la fois **(i)** un inventaire exhaustif des cas d’utilisation aligné sur le diagramme de la section 2.4 et sur le code livré, et **(ii)** des fiches détaillées (acteur, préconditions, postconditions, nominal, variantes, exceptions) pour les cas transverses les plus structurants. Le modèle de fiche est rappelé en **Annexe** (tableau 4.3).

Complétude et contraintes de volume. Le plan version 2 appelle une description textuelle très détaillée pour chaque cas d’utilisation. Pour rester lisible tout en garantissant une **traçabilité complète du périmètre fonctionnel**, ce mémoire retient une **double entrée** : des tableaux d’inventaire couvrant l’ensemble des capacités métier (§2.5.1), et des fiches détaillées sur un sous-ensemble représentatif au format annexe ; les autres cas suivent explicitement le même canevas (tableau 4.3) et sont couverts fonctionnellement par le code et l’inventaire.

2.5.1 Inventaire exhaustif des cas d’utilisation

Les tableaux 2.1 à 2.5 recensent, par paquetage métier, les capacités exposées par l’application (front-end React, API REST /api/*, webhooks /api/webhooks, orchestration /internal/* protégée, files BullMQ/Redis). Ils constituent la trace « exhaustive » attendue par le plan (version 2) au sens **fonctionnel** ; le §2.5.2 approfondit **cinq** cas transverses au format annexe (tableaux 2.6–2.10), tandis que le chapitre 3 illustre **sept** enchaînements critiques par couples diagramme de séquence / classes participantes, alignés sur le code.

TABLE 2.1 – Inventaire des cas d'utilisation — paquetage **Comptes**

Cas d'utilisation	Acteurs / surfaces logicielles
Inscription et vérification du compte (OTP et/ou lien e-mail)	Visiteur → Utilisateur; /api/auth; SMTP
Connexion, déconnexion, rotation des jetons (<i>refresh</i>)	Utilisateur, Administrateur; /api/auth; cookies HttpOnly
Mot de passe oublié / réinitialisation sécurisée	Visiteur/Utilisateur; /api/auth; SMTP
Consultation et mise à jour du profil (préférences de risque, catégories)	Utilisateur; /api/users
Suppression de compte (contraintes si investissement actif)	Utilisateur; /api/users

TABLE 2.2 – Inventaire des cas d'utilisation — paquetage **Projets**

Cas d'utilisation	Acteurs / surfaces logicielles
Exploration, recherche et consultation d'un projet public	Visiteur, Utilisateur; /api/projects/public, /api/projects/search, optionalAuth
Création, modification, archivage d'un brouillon (DRAFT)	Porteur de projet; /api/projects
Soumission à l'analyse IA (AWAITING_AI) et suivi des relances	Porteur; n8n/BullMQ; /internal/run-risk-analysis (routes internes authentifiées)
Décision administrative (UNDER_REVIEW → APPROVED/REJECTED), publication (ACTIVE)	Administrateur; /api/admin/projects
Suspension / réactivation, relance d'analyse IA	Administrateur; /api/admin/projects
Révoquer une approbation (APPROVED → REJECTED) pour demander des corrections	Administrateur; /api/admin/projects/.../revoke-approval

TABLE 2.3 – Inventaire des cas d'utilisation — paquetage Contributions & finance

Cas d'utilisation	Acteurs / surfaces logicielles
Initier une contribution, paiement simulé, <i>webhook</i> signé et idempotent	Investisseur, passerelle; /api/investments, /api/webhooks
Annulation pendant la fenêtre de grâce (CANCELLING/CANCELLED)	Investisseur; /api/investments/.../cancel
Liste « mes investissements », relance de paiement, suivi des transactions	Investisseur; collections investments, transactions
Remboursements (REFUNDED), anti-sur-financement, relances admin d'échecs	Système, administrateur; services métier + files
Saisie chiffrée des coordonnées bancaires (AES-256-GCM)	Porteur de projet; PUT /api/payouts/:id/bank-details
Demande de retrait (<i>payout</i>) et validation administrative	Porteur, administrateur; /api/payouts, /api/admin

TABLE 2.4 – Inventaire des cas d'utilisation — paquetage Échanges & confiance

Cas d'utilisation	Acteurs / surfaces logicielles
Commentaires sur projet public (publication, suppression auteur)	Utilisateur; /api/projects/.../comments
Signalement projet ou commentaire; consultation « mes signalements »	Utilisateur; /api/reports
Traitement des signalements (RESOLVED/DISMISSED)	Administrateur; /api/admin/reports
Notifications in-app (GET/PUT /api/notifications, userId=session, pagination page/limit ≤ 30, tri antéchronologique); e-mails critiques via BullMQ	Tous rôles; workers SMTP
Supervision des envois SMTP en échec et relance manuelle	Administrateur; /api/admin/failed-notifications; IHM /admin/email-failures

Cas d'utilisation	Acteurs / surfaces logicielles
Recommandations (score déterministe : catégories, riskLevel, progression, récence)	Utilisateur; /api/recommendations
Chatbot d'aide contextualisé sur un projet (LLM)	Utilisateur; POST /api/projects/:id/chat

TABLE 2.5 – Inventaire des cas d'utilisation — paquetage Administration

Cas d'utilisation	Acteurs / surfaces logicielles
Désactivation / réactivation d'utilisateurs	Administrateur; /api/admin/users
Modération des commentaires (masquage / rétablissement)	Administrateur; /api/admin/comments
Supervision des files d'échec (e-mails, payouts, remboursements, annulations)	Administrateur; collections failed* + actions de relance

2.5.2 Fiches détaillées (format annexe)

Les tableaux 2.6 à 2.10 détaillent cinq cas transverses (création de projet, contribution, validation administrative, recommandations, signalement). Les autres lignes des tableaux 2.1 à 2.5 suivent le même canevas en annexe ou en fiches séparées si le jury l'exige.

Cas d'utilisation	Créer et soumettre un projet
Acteur	Utilisateur (porteur de projet)
Précondition	L'utilisateur est authentifié
Postcondition	Projet persisté ; statut DRAFT puis AWAITING_AI après soumission ; analyse IA planifiée
Scénario nominal	1. L'utilisateur renseigne titre, description, catégorie, objectif (<code>fundingGoal</code> ≥ 1), fenêtre temporelle (<code>startAt</code> , <code>deadline</code>), médias éventuels.
	2. Le système enregistre un projet en DRAFT.
	3. L'utilisateur déclenche la soumission : passage en AWAITING_AI, création d'un job (trace <code>aiJobId</code> , compteurs de relance).
	4. Le workflow n8n/LLM produit ou enrichit <code>project.aiAnalysis</code> (rapport + <code>score/riskLevel</code>).
	5. Si aucune règle bloquante n'est déclenchée, le projet passe en UNDER_REVIEW et l'administration est notifiée.
Scénarios alternatifs	1. Auto-rejet budgétaire : heuristique <code>riskHeuristicService</code> — si l'écart entre l'objectif et l'estimation des besoins extraits de la description atteint ou dépasse 30 % (sévérité BLOCK), passage immédiat en REJECTED avec motif structuré (<code>/internal/workflow</code>).
	2. Échec ou indisponibilité du pipeline IA : le projet reste en AWAITING_AI ; relances automatiques (<code>aiAutoRetryCount</code> , <code>aiAnalysisRetries</code> bornées).
Exceptions	1. Validation serveur (champs obligatoires, dates, médias).
	2. <code>fundingGoal</code> invalide (< 1) : rejet 400.

TABLE 2.6 – Description textuelle du cas « Créer et soumettre un projet »

Cas d'utilisation	Soutenir un projet (paiement simulé)
Acteur	Utilisateur (investisseur), Passerelle de paiement
Précondition	Utilisateur authentifié ; projet ACTIVE ; montant $\geq$ minimum applicatif (100 TND côté API)
Postcondition	Investissement INITIATED puis SUCCESS si <i>webhook</i> SUCCEDED ; incrément atomique de currentFunding
Scénario nominal	1. L'utilisateur saisit un montant $\geq$ 100 TND.
	2. Le système crée un investissement (INITIATED) et une transaction (PENDING , couple provider+providerPaymentId unique).
	3. Le système renvoie une URL de paiement simulée.
	4. La passerelle appelle le <i>webhook</i> signé.
	5. Le backend vérifie signature + idempotence.
	6. Transaction SUCCEDED , investissement SUCCESS , mise à jour du total collecté (garde-fou anti-sur-financement).
Scénarios alternatifs	1. Paiement refusé : transaction FAILED / CANCELLED , investissement FAILED .
	2. Sur-financement : statuts REFUNDED , remontées admin pour relance éventuelle.
	3. Annulation utilisateur dans la fenêtre de grâce (CANCELLING $\rightarrow$ CANCELLED).
Exceptions	1. Projet indisponible ou investisseur administrateur (règle requireNotAdmin).
	2. Montant hors bornes (<100 TND).

TABLE 2.7 – Description textuelle du cas « Soutenir un projet »

Cas d'utilisation	Valider / rejeter un projet
Acteur	Administrateur
Précondition	Authentification admin ; projet en UNDER_REVIEW (sauf scénario de révocation)
Postcondition	Projet APPROVED ou REJECTED ; journalisation ; notifications créateur
Scénario nominal	1. L'administrateur consulte la file des projets en revue.
	2. Il analyse le rapport IA (aiAnalysis.report, scores).
	3. Il émet une décision documentée (APPROVED/REJECTED).
	4. Le service adminProjectService applique la transition autorisée (projectLifecycle).
	5. Notifications in-app + e-mail au créateur.
Scénarios alternatifs	1. Projet REJECTED : le créateur peut corriger et resoumettre (DRAFT → cycle).
	2. Révocation d'approbation : depuis APPROVED, l'administrateur appelle /api/admin/projects/:id/revoke-approval ; retour REJECTED avec demande de corrections.
Exceptions	1. Identifiant inconnu ou droits insuffisants.
	2. Transition interdite (ex. publication sans approbation).

TABLE 2.8 – Description textuelle du cas « Valider / rejeter un projet »

Cas d'utilisation	Voir les projets recommandés
Acteur	Utilisateur (investisseur)
Précondition	Utilisateur authentifié
Postcondition	Liste de projets ACTIVE non archivés, triée par score interne
Scénario nominal	1. L'utilisateur ouvre l'écran « recommandations ».
	2. Le backend charge <code>profile.preferredCategories</code> et <code>profile.riskPreference</code> .
	3. <code>recommendationService</code> calcule un score déterministe par projet (adéquation catégorie, cohérence avec <code>aiAnalysis.riskLevel</code> , progression, récence).
	4. Les projets sont triés par score décroissant (limite paramétrable, défaut 8).
Scénarios alternatifs	1. Aucun projet ACTIVE : liste vide + message.
	2. Risque ou catégorie manquants : pénalités neutres dans le score (implémentation actuelle).
Exceptions	1. Session expirée (401).

TABLE 2.9 – Description textuelle du cas « Voir les projets recommandés »

Cas d'utilisation	Signaler un contenu (projet ou commentaire)
Acteur	Utilisateur
Précondition	Utilisateur authentifié ; cible existante
Postcondition	Rapport stocké avec statut PENDING (énumération FRAUD, INAPPROPRIATE_CONTENT, SPAM, OTHER)
Scénario nominal	1. L'utilisateur ouvre le formulaire de signalement.
	2. Il choisit le type, rédige la description, lie éventuellement un commentaire.
	3. Le backend persiste le signalement et notifie l'administration.
Scénarios alternatifs	1. Consultation de l'historique « mes signalements ».
	2. Administrateur : résolution (RESOLVED) ou classement sans suite (DISMISSED).
Exceptions	1. Champs requis manquants.
	2. Signalement redondant (contraintes d'unicité <code>reports</code> côté base).

TABLE 2.10 – Description textuelle du cas « Signaler un contenu »

2.6 Conclusion

Ce chapitre a précisé la délimitation du système en tant que « boîte noire », recensé **six** acteurs pertinents dans le contexte (Visiteur, Utilisateur authentifié, Administrateur, passerelle de paiement de démonstration, service d'analyse de risque orchestré, service SMTP) et ordonné les besoins en **cinq** paquetages métier (Comptes, Projets, Contributions & finance, Échanges & confiance, Administration). Les tableaux 2.1 à 2.5 consolident l'inventaire exhaustif des cas d'utilisation, en stricte continuité avec l'implémentation ; les fiches 2.6–2.10 en offrent, pour des cas transverses majeurs, une illustration au format annexe. Le chapitre 3 poursuit la démarche par la traduction de ces besoins en conception UML et par la description du **modèle de données MongoDB**.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre transpose les besoins recueillis au chapitre 2 en une conception UML alignée sur le plan (version 2). Il expose d'abord la **réalisation conceptuelle** des cas d'utilisation les plus structurants au travers de diagrammes de séquence construits selon la notation retenue (acteurs, **IHM**, **contrôleur** et classes métier effectivement sollicitées). Chaque séquence est complétée par un diagramme de **classes participantes** qui en fixe la contrepartie statique. Sont ensuite présentés le **modèle du domaine** (diagramme de classes global), l'**architecture du futur logiciel** puis le **modèle de données MongoDB**. Une conclusion récapitule les choix de conception et annonce l'implémentation.

3.2 Réalisation conceptuelle des cas d'utilisation

Les diagrammes de séquence suivants décrivent le flux d'interactions entre les acteurs et les objets du système pour un ensemble de cas d'utilisation représentatifs identifiés au chapitre 2. Chaque séquence est complétée par un diagramme de classes participantes afin d'assurer la cohérence entre la vue dynamique (interactions) et la vue statique (classes mobilisées). Ces diagrammes sont alignés sur les règles métier et sur l'implémentation livrée.

Choix des scénarisations UML. Le référentiel académique peut exiger une interaction par cas d'utilisation ; compte tenu du nombre de cas listés aux tableaux 2.1–2.5, cette section présente en priorité les **enchaînements les plus structurants** (contributions et financement, cycle de projet,

conformité administrative, retrait). Elle illustre la démarche de conception et garantit la **mise en cohérence** avec le modèle du domaine (figure 3.15) ; les autres comportements reposent sur les mêmes classes et contrats d'API documentés au chapitre 2.

Cas retenus. Annulation de contribution, création et soumission d'un projet, investissement (paiement démo + *webhook*), validation/rejet d'un projet (administration), fourniture de coordonnées bancaires, approbation d'un retrait (administration) et mise à jour de l'état d'un projet (suspension/réactivation).

Structure de chaque sous-section : diagramme de séquence puis son commentaire ; diagramme de classes participantes puis son commentaire.

3.2.1 Annuler une contribution

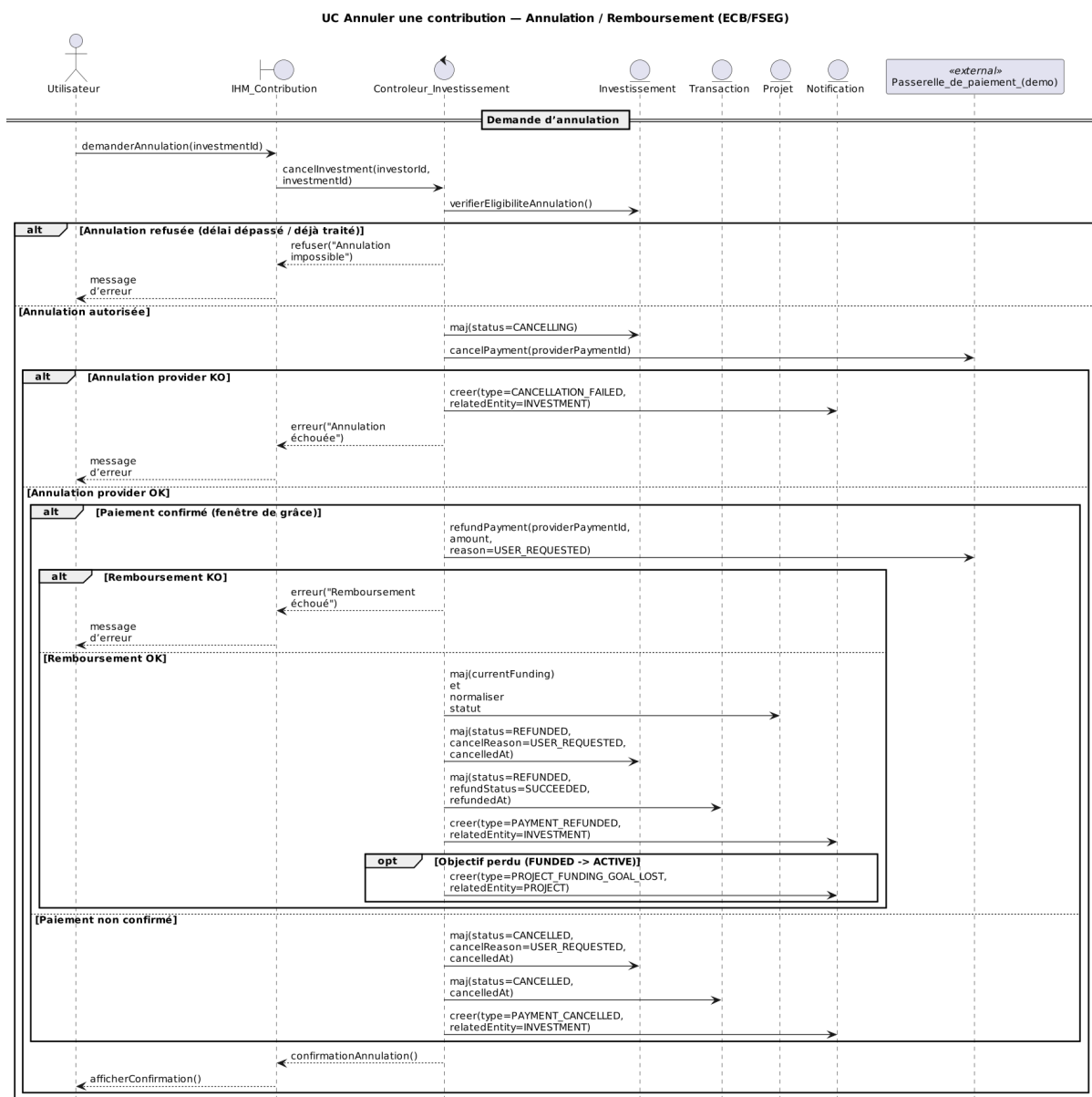


FIGURE 3.1 – Diagramme de séquence – Annuler une contribution

Diagramme de séquence (figure 3.1) : il illustre l'ordre chronologique des interactions : l'investisseur initie l'annulation via l'interface, le contrôleur vérifie les règles métier (délai de grâce, état de l'investissement et de la transaction), sollicite la passerelle de paiement de démonstration si nécessaire, met à jour l'investissement et la transaction, puis notifie l'utilisateur du succès ou de l'échec.

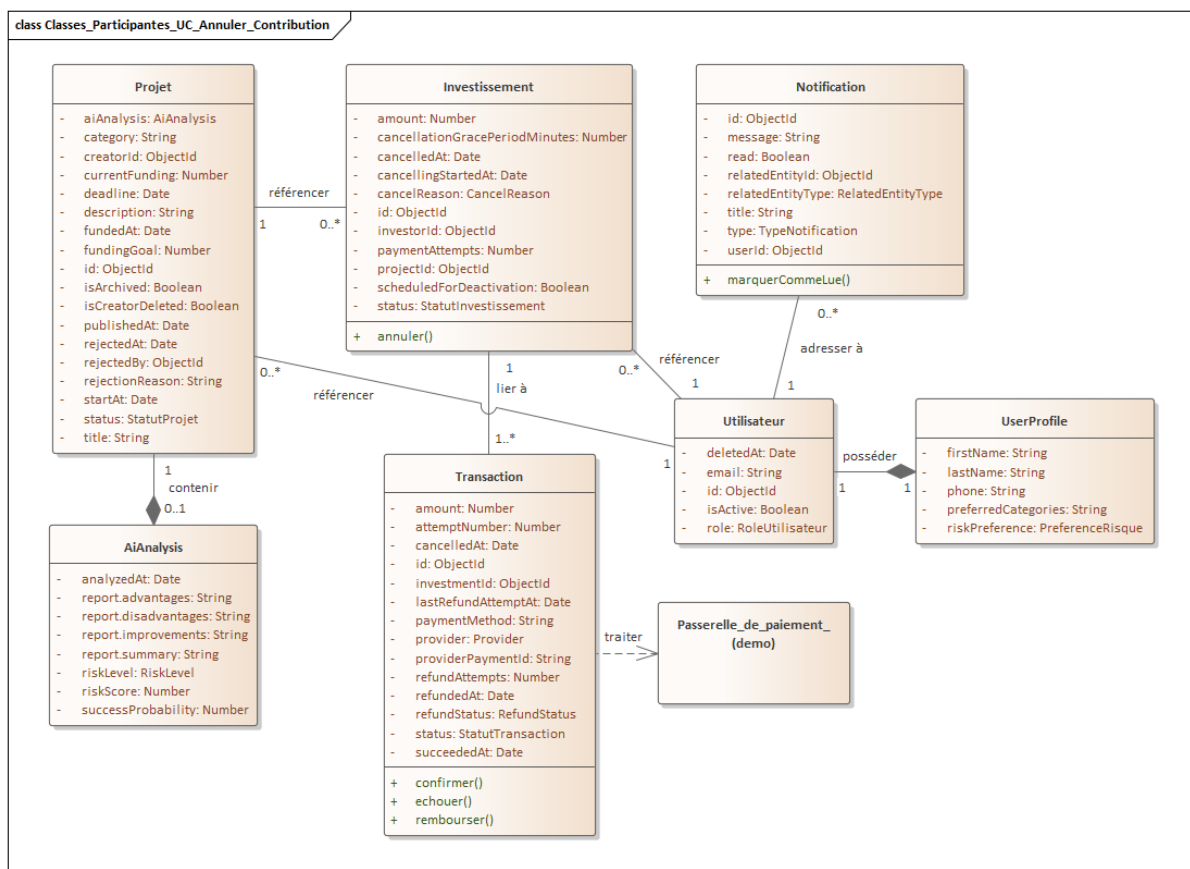


FIGURE 3.2 – Diagramme de classes participantes – Annuler une contribution

Diagramme de classes participantes (figure 3.2) : il fixe la vue statique associée à la séquence ci-dessus ; il recense les entités métier mobilisées (Utilisateur, Projet, Investissement, Transaction, Notification) ainsi que la passerelle externe, avec les attributs et énumérations utiles au cas (statuts d'investissement, de transaction, motifs d'annulation, etc.).

3.2.2 Créer et soumettre un projet

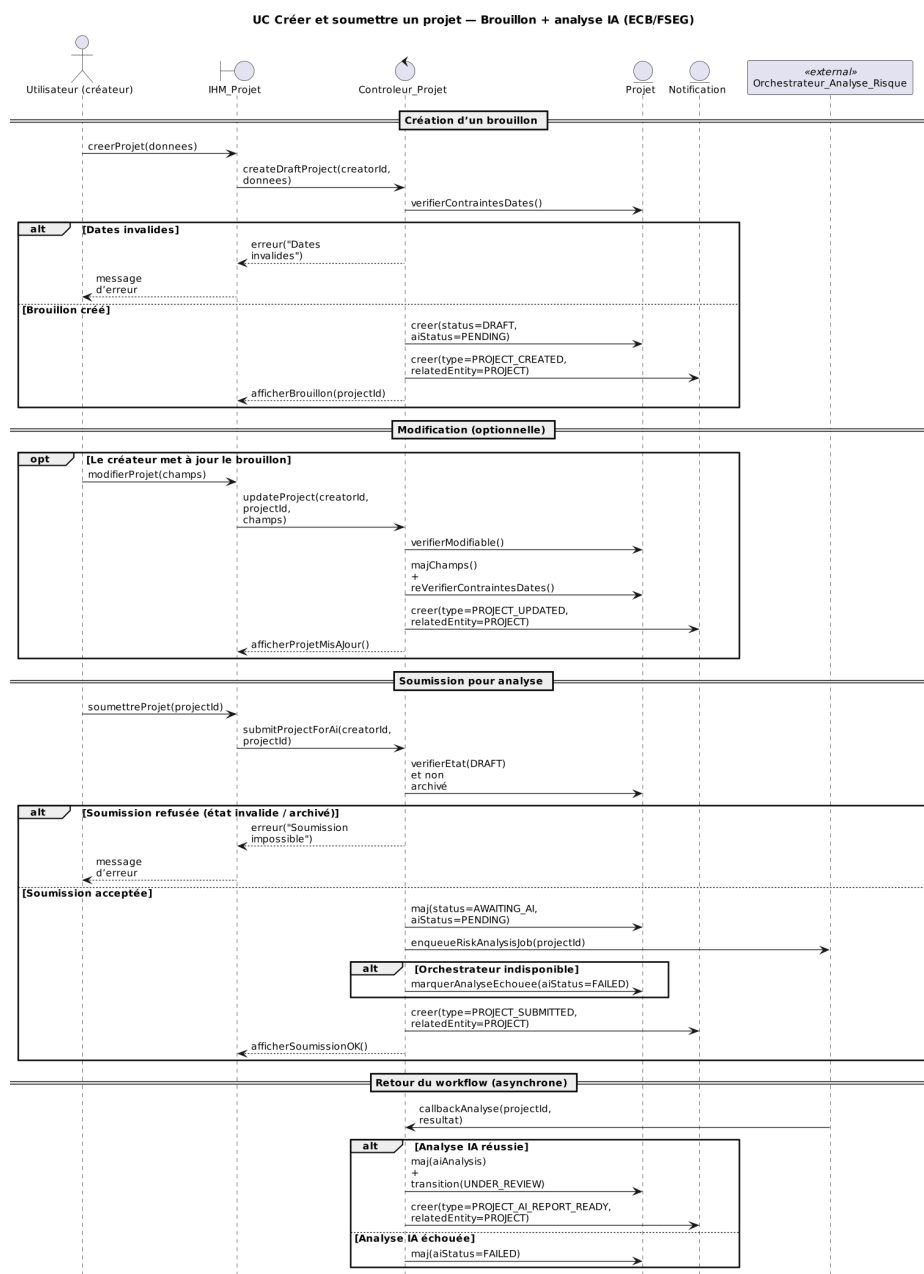


FIGURE 3.3 – Diagramme de séquence – Créer et soumettre un projet

Diagramme de séquence (figure 3.3) : il détaille le scénario de création puis de soumission : le porteur renseigne le projet via l'IHM, le contrôleur persiste le brouillon, déclenche l'analyse de risque (orchestration externe), met à jour les états du projet (DRAFT, AWAITING_AI, UNDER_REVIEW, etc.) et notifie le créateur des transitions importantes.

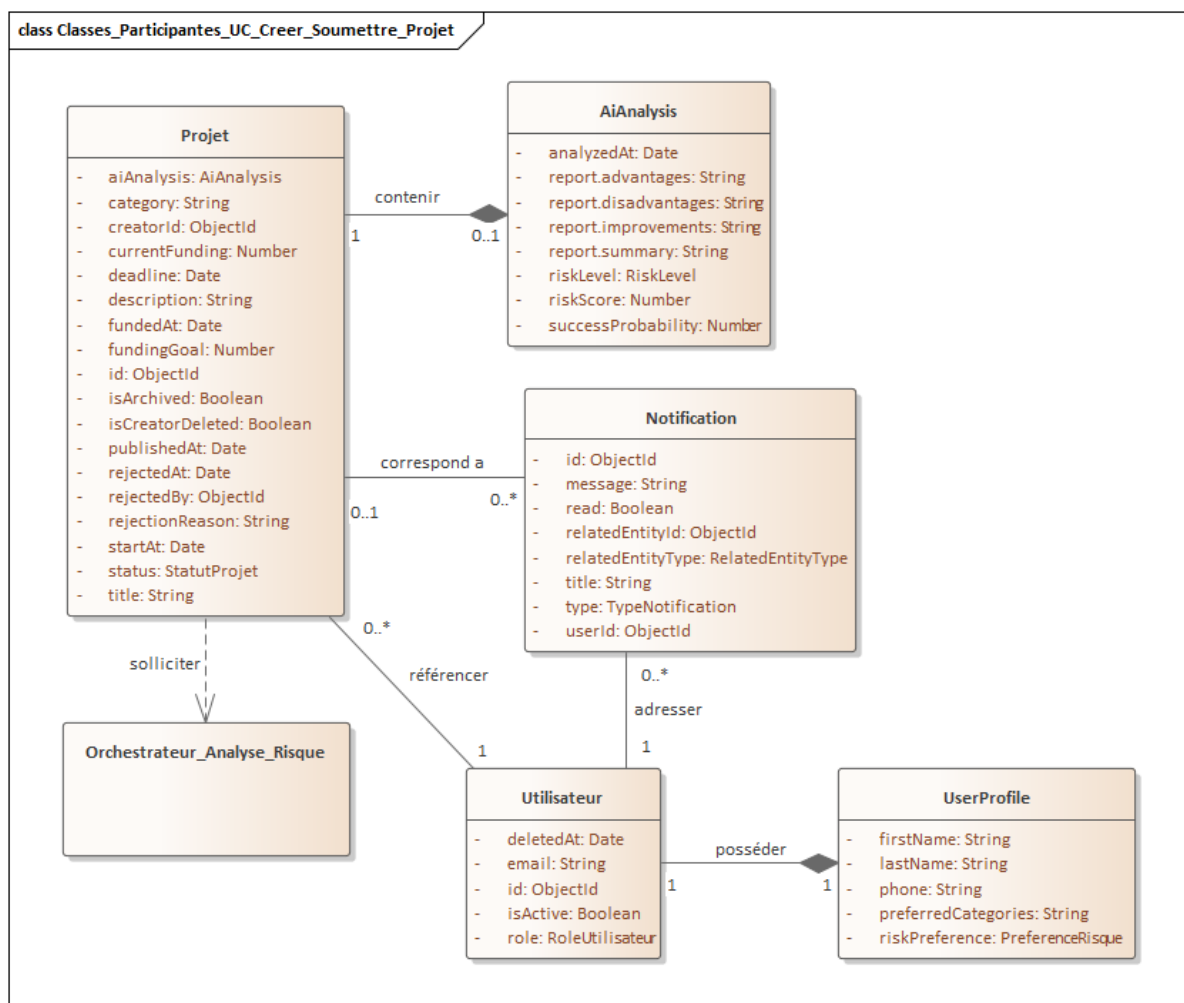


FIGURE 3.4 – Diagramme de classes participantes – Créer et soumettre un projet

Diagramme de classes participantes (figure 3.4) : il montre les classes du domaine concernées par ce flux (Utilisateur, profil, Projet, sous-objet d’analyse IA, Notification) et leurs attributs alignés sur le schéma applicatif; il fait le lien entre les messages de la séquence et la structure de données persistée en base.

3.2.3 Investir dans un projet (paiement démo + *webhook*)

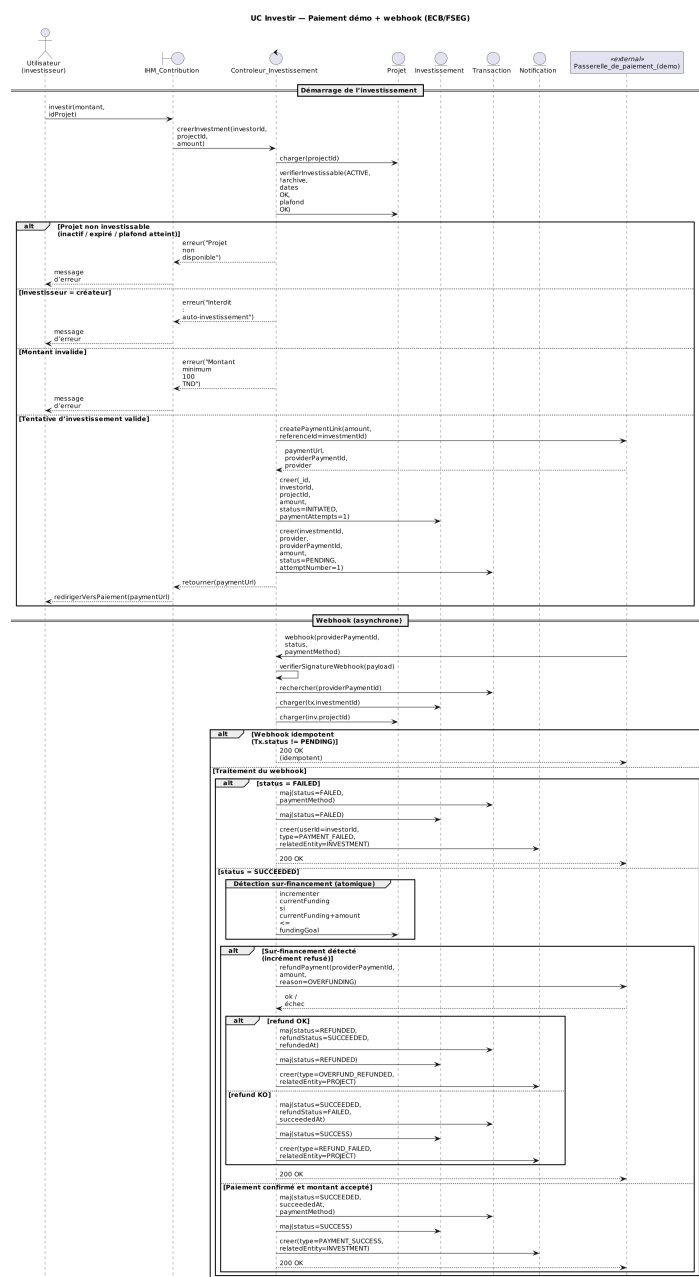


FIGURE 3.5 – Diagramme de séquence – Investir (paiement démo + *webhook*)

Diagramme de séquence (figure 3.5) : il met en évidence le caractère **asynchrone** du paiement démo : l'investisseur initie le flux côté application, la passerelle confirme ou refuse le paiement, puis le *webhook* rappelle le backend pour finaliser la transaction, promouvoir l'investissement au statut réussi et déclencher les notifications associées.

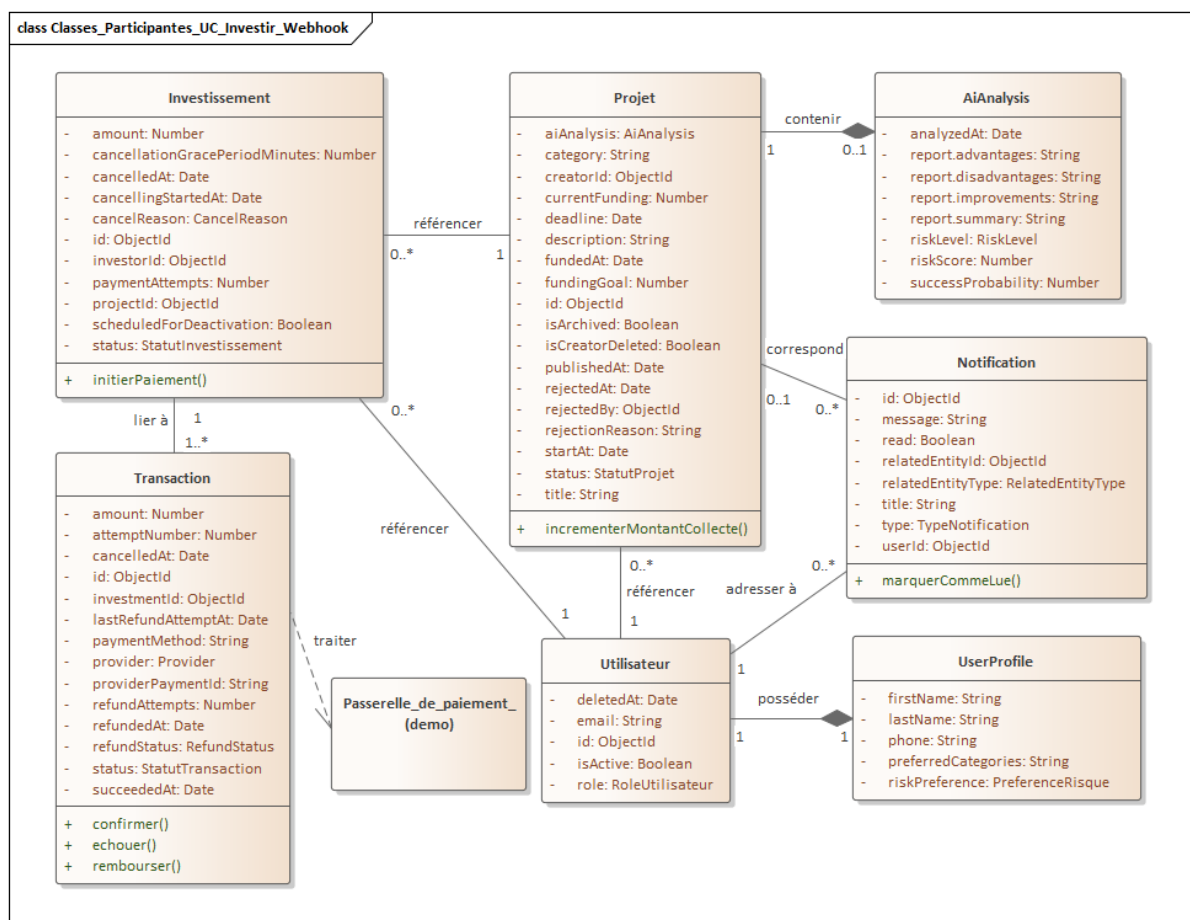


FIGURE 3.6 – Diagramme de classes participantes – Investir dans un projet (paiement démo + *webhook*)

Diagramme de classes participantes (figure 3.6) : il explicite les entités et attributs impliqués (Utilisateur investisseur, Projet, Investissement, Transaction, Notification, passerelle) et permet de relier chaque message de la séquence à une structure de données concrète (montants, statuts, identifiants fournisseur, liens vers le projet).

3.2.4 Valider un projet (Admin)

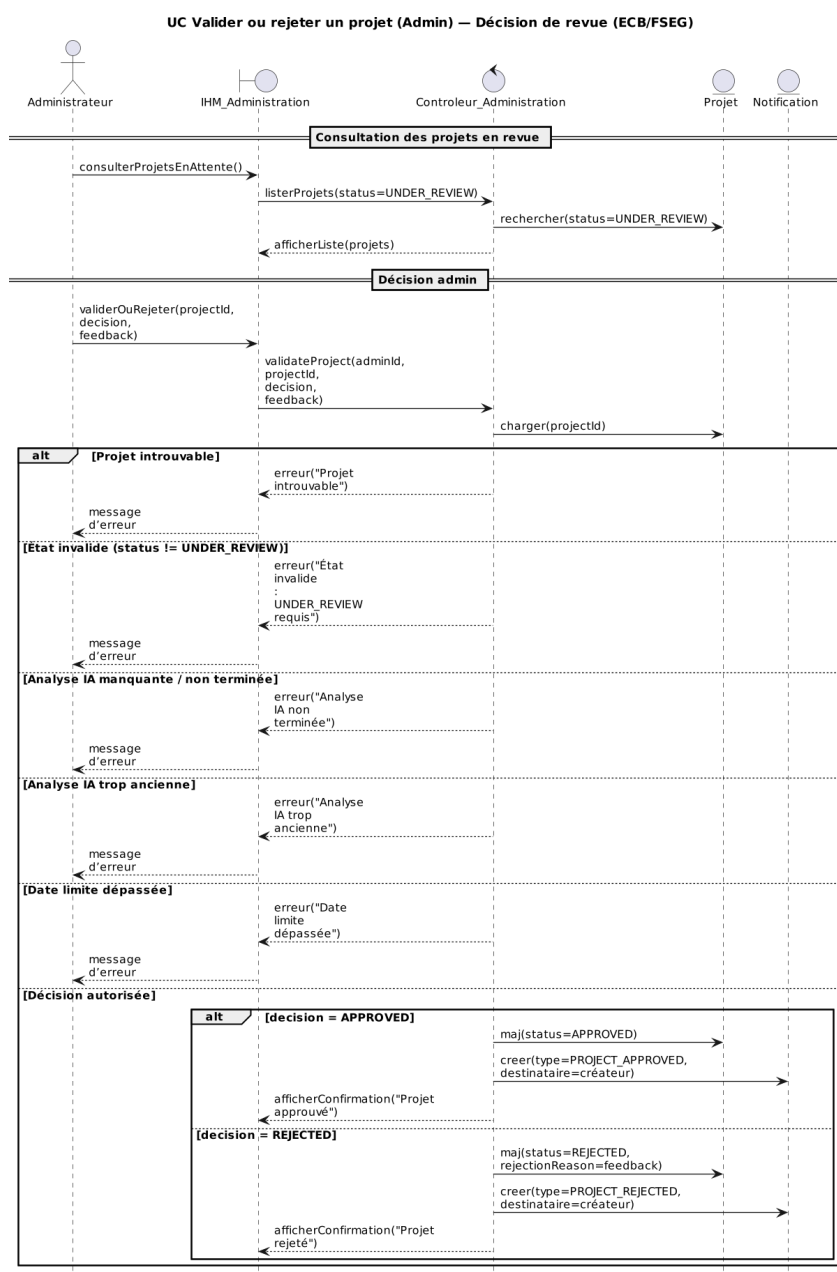


FIGURE 3.7 – Diagramme de séquence – Valider un projet

Diagramme de séquence (figure 3.7) : il décrit le parcours administrateur : chargement d'un projet en revue, contrôle des éléments de décision (y compris synthèse d'analyse lorsqu'elle est disponible), enregistrement du verdict (approbation ou rejet avec motif), mise à jour du cycle de vie du projet et envoi de notifications au créateur.

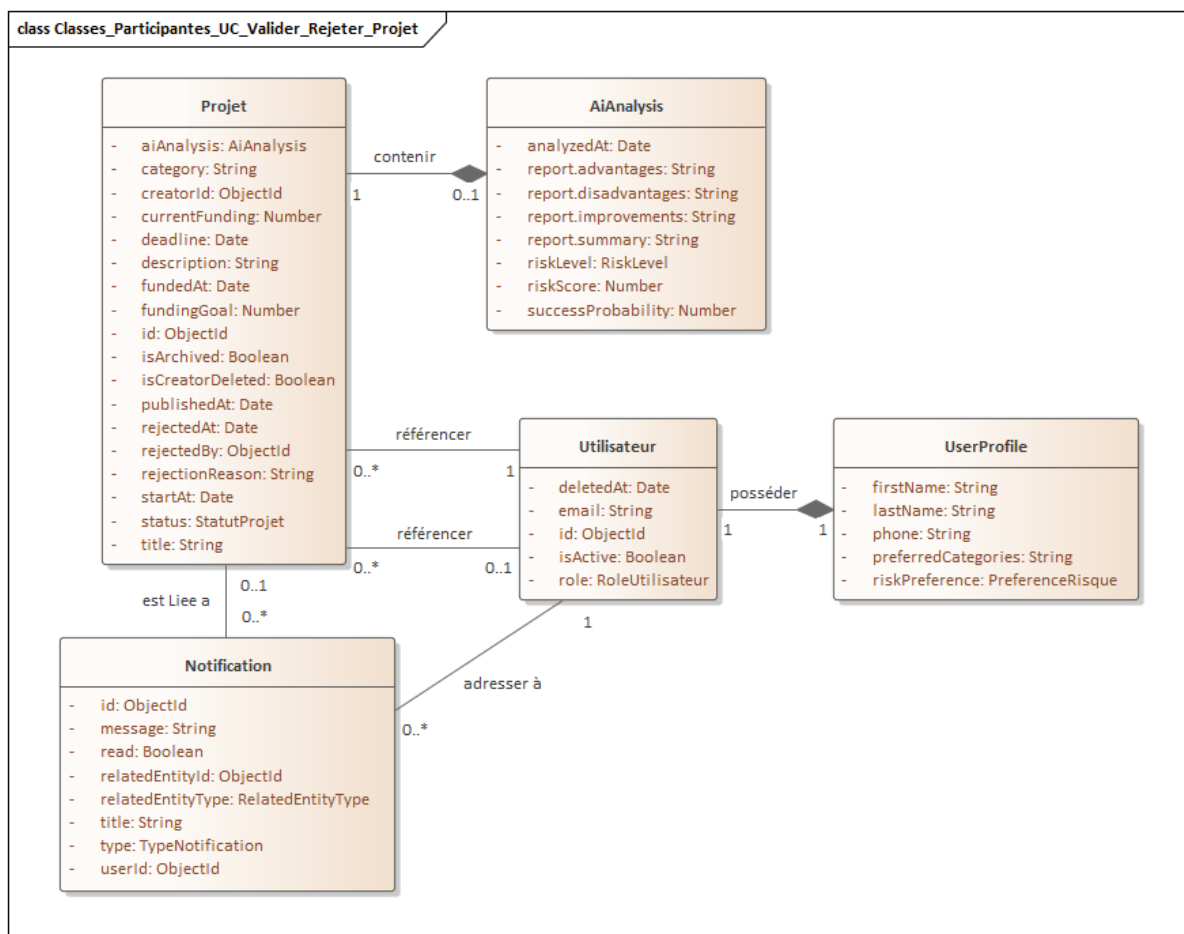


FIGURE 3.8 – Diagramme de classes participantes – Valider ou rejeter un projet (Admin)

Diagramme de classes participantes (figure 3.8) : il complète la vue en montrant les classes métier mobilisées (**Utilisateur**, **Projet**, bloc d'analyse IA, **Notification**) et matérialise les données que la séquence lit ou modifie (statut du projet, raison de rejet, rattachement des notifications).

3.2.5 Fournir les coordonnées bancaires

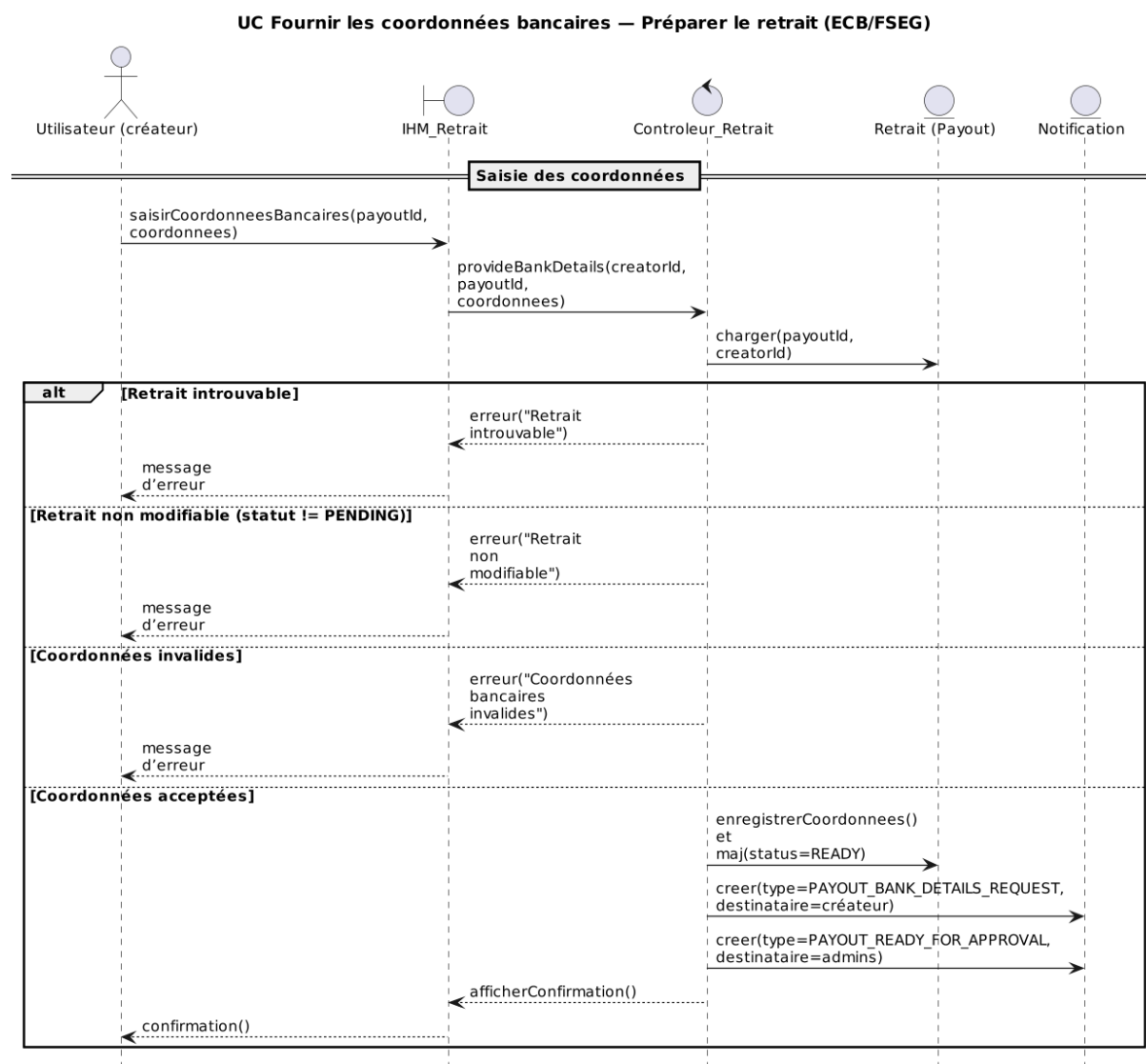


FIGURE 3.9 – Diagramme de séquence – Fournir les coordonnées bancaires

Diagramme de séquence (figure 3.9) : il montre comment le créateur fournit les informations bancaires encadrées par l’application, comment le service métier met à jour l’objet Payout (coordonnées, date de saisie, passage à l’état prêt pour validation) et comment des notifications sont émises vers le créateur et/ou l’administration.

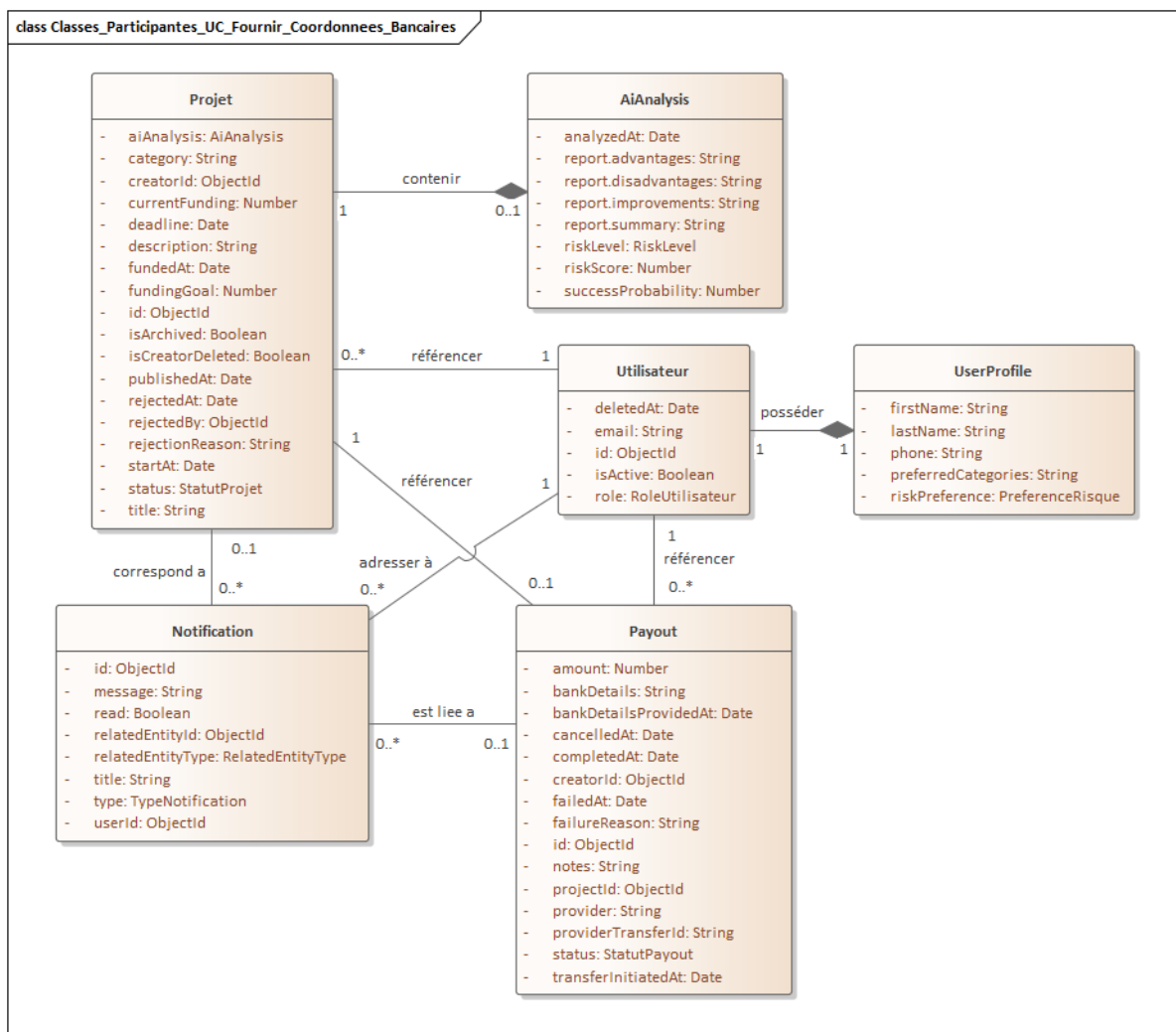


FIGURE 3.10 – Diagramme de classes participantes – Fournir les coordonnées bancaires

Diagramme de classes participantes (figure 3.10) : il présente les entités en jeu (Utilisateur, Projet, Payout, Notification) avec les champs pertinents pour ce cas et relie visuellement les messages de la séquence aux agrégats persistés en MongoDB.

3.2.6 Approuver un retrait (Admin)

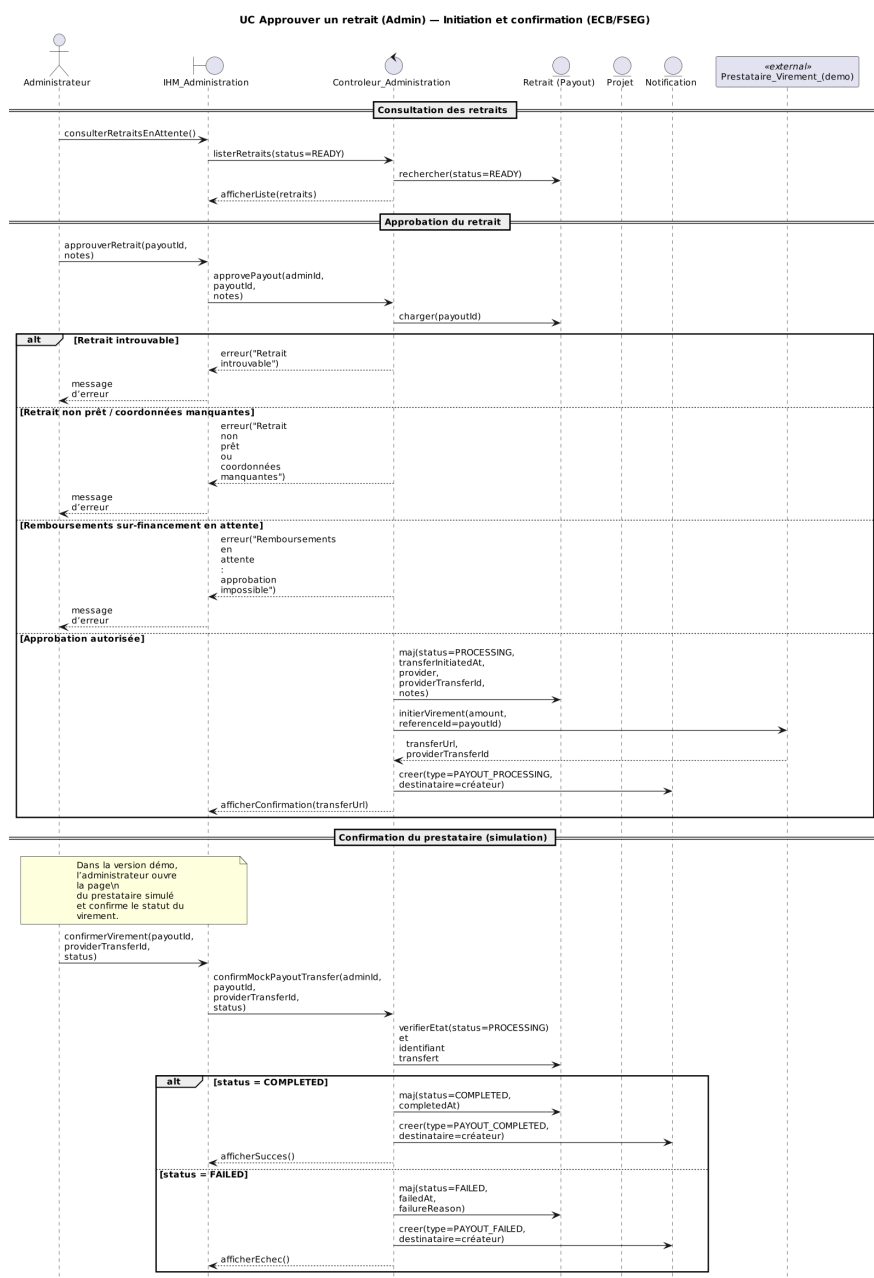


FIGURE 3.11 – Diagramme de séquence – Approuver un retrait (Admin)

Diagramme de séquence (figure 3.11) : il ordonne les actions d’approbation : contrôle des préconditions (retrait READY, coordonnées présentes), appel au prestataire de virement de démonstration, mise à jour des statuts temporels du Payout (PROCESSING, COMPLETED ou échec) et notification au créateur.

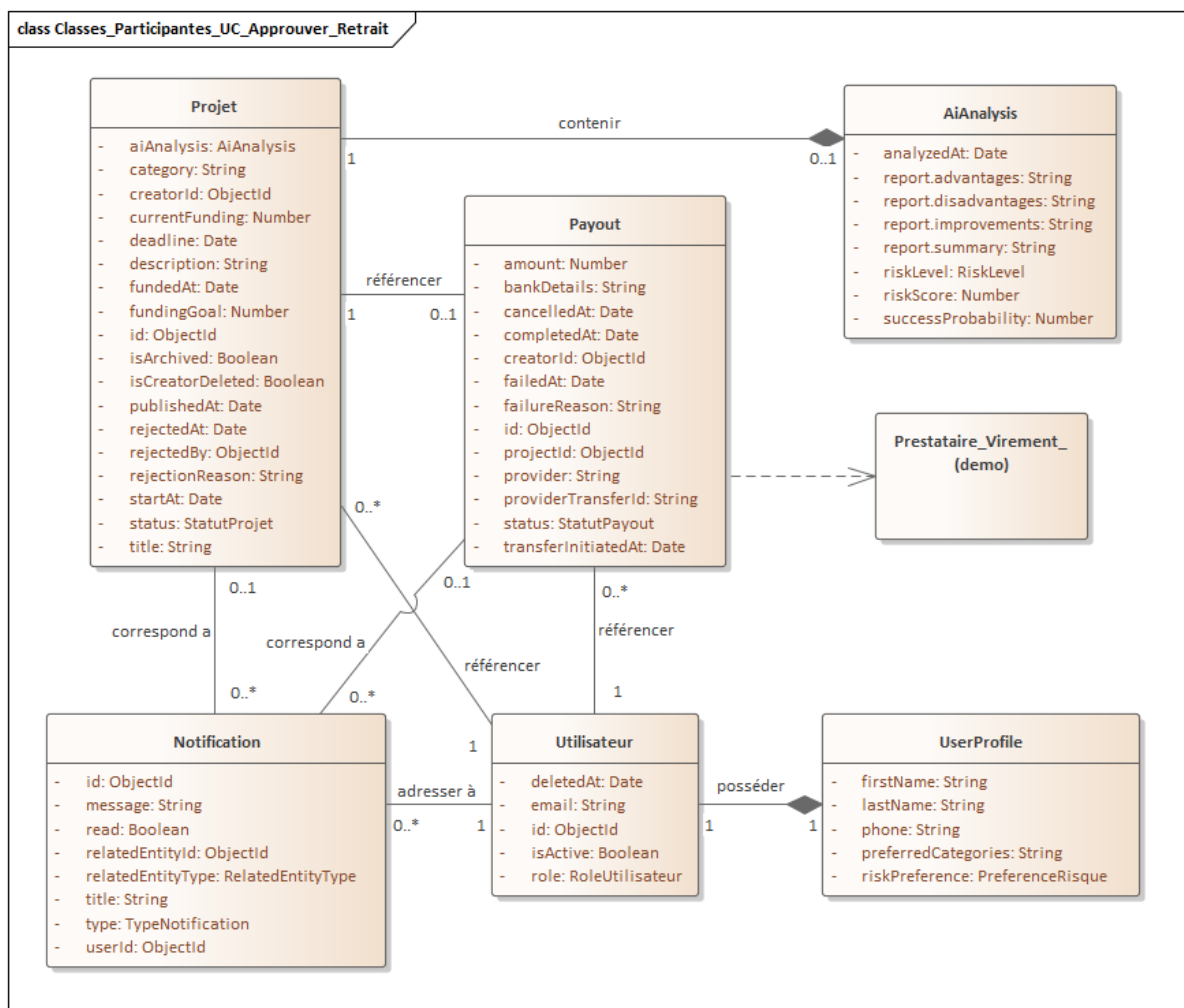


FIGURE 3.12 – Diagramme de classes participantes – Approuver un retrait (Admin)

Diagramme de classes participantes (figure 3.12) : il identifie les classes du domaine et l'acteur externe (Projet, Payout, Utilisateur, Notification, prestataire) et précise quelles données sont lues ou écrites à chaque étape visible sur la séquence.

3.2.7 Mise à jour de l'état d'un projet (Admin)

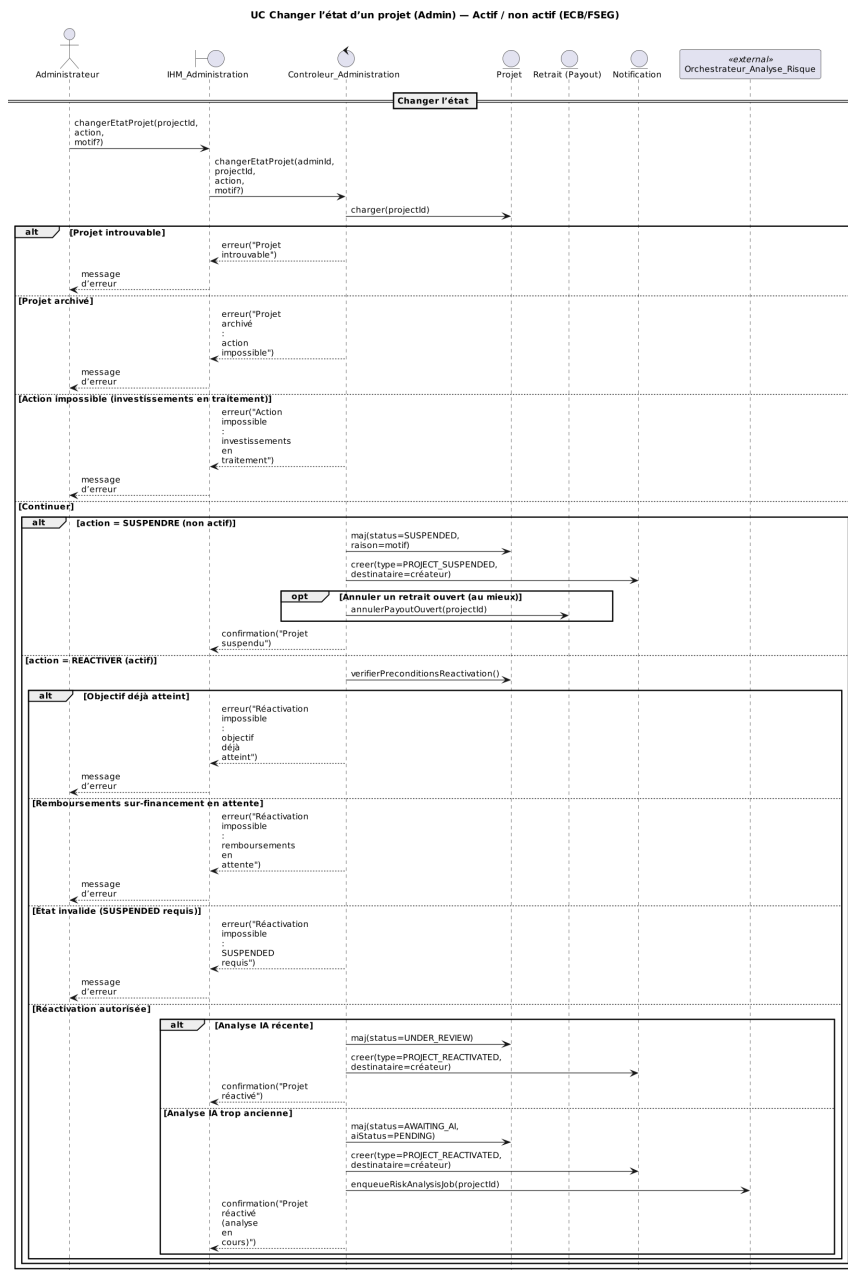


FIGURE 3.13 – Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)

Diagramme de séquence (figure 3.13) : il décrit la suspension ou la réactivation d'un projet par l'administrateur : sélection du projet, vérification des transitions autorisées depuis l'état courant, persistance du nouveau statut (SUSPENDED ou retour vers un état actif) et envoi de notifications explicites (PROJECT_SUSPENDED, PROJECT_REACTIVATED, etc.).

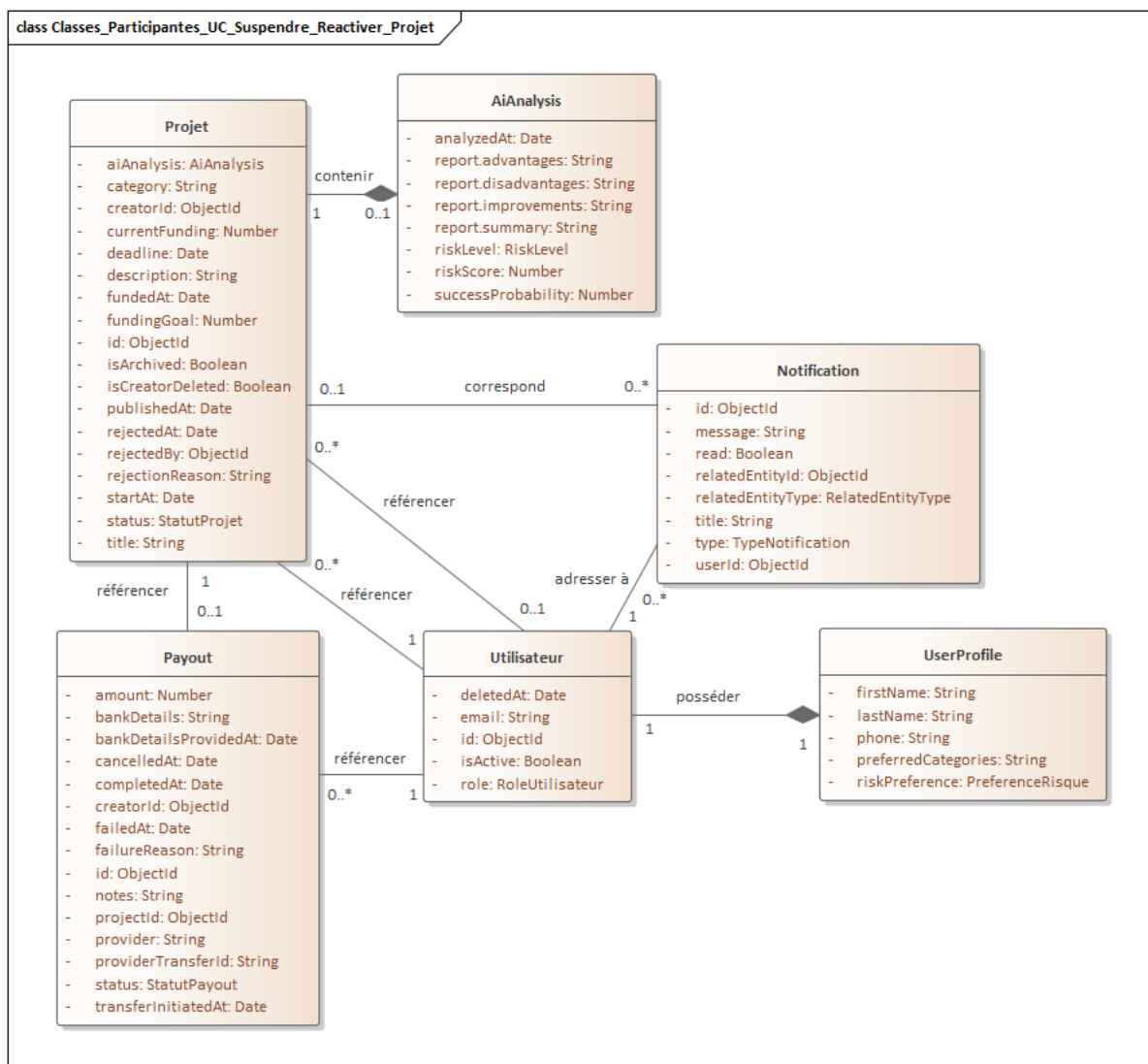


FIGURE 3.14 – Diagramme de classes participantes – Suspendre ou réactiver un projet (Admin)

Diagramme de classes participantes (figure 3.14) : il synthétise les entités impactées (Utilisateur, Projet, analyse IA éventuelle, Notification) et clarifie quels attributs de projet et de notification portent la trace métier visible dans la séquence.

3.3 Modèle du domaine

Lecture du diagramme. Le diagramme de classes qui précède matérialise le **modèle du domaine** de la plateforme : il explicite les entités métier centrales (**Utilisateur, Projet, Investissement, Transaction, Retrait, Notification, Commentaire, Signalement**) avec leurs attributs, leurs



FIGURE 3.15 – Diagramme de classes du domaine (modèle du domaine)

opérations et leurs relations assorties de multiplicités. Les énumérations portant sur le rôle, les statuts ou les types de notification rendent lisibles les cycles de vie et les règles de gestion correspondantes. Trois invariants y sont rappelés de manière synthétique : (i) unicité de l'adresse électronique ; (ii) plafonnement de la collecte par l'objectif de financement ; (iii) limitation à un retrait par projet. Ce modèle fait office de référence pour les développements ultérieurs relatifs à l'architecture et au **modèle de données MongoDB**.

3.4 Architecture du futur logiciel

3.4.1 Vue d'ensemble en couches

L'application suit une architecture client-serveur à trois couches, déployée sur un cloud commercial :

- **Couche présentation** : application React (SPA) servie via Vite, communiquant avec le backend par appels REST sécurisés (cookies HttpOnly).
- **Couche métier** : Node.js + Express, qui expose les points d'accès API, applique les règles de validation (Zod), orchestre les services internes et gère les transactions MongoDB.
- **Couche données** : MongoDB (replica set requis pour les transactions multi-documents).

Les services internes (`projectService`, `investmentService`, `payoutService`, `adminProjectService`, `adminUserService`, `accountService`, `notificationService`, `recommendationService`, `reportService`, `authService`, `workflowInternalService`, `geminiRiskService`, `geminiChatService`) sont des modules distincts au sein du backend. Ils sont assemblés par l'*Express router* et leurs traitements asynchrones sont délégués à des files BullMQ/Redis (e-mails, retentatives) ou à des workflows n8n (analyse de risque); le chatbot est traité par l'API Express (POST `/api/projects/:id/chat`) avec appel LLM côté serveur.

3.4.2 Intégration de n8n

n8n est un outil d'orchestration visuelle de workflows utilisé pour découpler la logique métier des services externes. Dans le projet :

- **Analyse de risque** : à la soumission d'un projet, le backend déclenche un workflow n8n qui interroge le service LLM, reçoit un rapport structuré, applique le seuil d'auto-rejet ($\text{gap} \geq 30\%$), puis met à jour le projet et notifie les acteurs concernés.
- **Chatbot d'aide sur projet** : les questions contextualisées sont traitées par la route POST

`/api/projects/:id/chat` (middleware d'authentification, *rate limiting* dédié), via le service serveur `geminiChatService` qui appelle directement le LLM ; **n8n n'intervient pas** sur ce flux dans la version livrée.

Les apports principaux sont les suivants : **découplage** (le backend n'expose, pour ce volet, qu'un point d'entrée de type *webhook* assorti d'une signature), **résilience** (politique de *retries* exponentiels 1/2/4 minutes, plafonnée à trois tentatives, avec bascule ultérieure vers `FailedWorkflowEvents`) et **évolutivité** (ajout de nouveaux workflows sans modification du noyau applicatif).

3.4.3 Composants asynchrones (BullMQ / Redis)

Les e-mails critiques (validation, contribution, paiement, retrait, désactivation) et les opérations de reprise sont gérés par des files BullMQ persistées dans Redis. Cette architecture absorbe les pics de charge et fournit un mécanisme uniforme de relance et de DLQ (`FailedRefund`, `FailedPayoutEvents`, `FailedCancellationEvents`). Les jobs récurrents (cron) sont gérés par le service interne `cronService` : expiration des projets et investissements, reprise des analyses bloquées, nettoyage des sessions et des jetons de rafraîchissement.

3.4.4 Vue logique en paquetages

Le diagramme de paquetages de la figure 3.16 décompose l'application en domaines fonctionnels cohérents avec le chapitre 2 (*Comptes, Projets, Contributions & finance, Échanges & confiance, Administration*) et distingue en outre la **couche applicative interne** (`/internal/*`, workflows et cron) ; il matérialise les dépendances vers les composants externes (*n8n, BullMQ/Redis, MongoDB*). Chaque bloc regroupe les services et routes qui portent une même responsabilité métier ou technique.

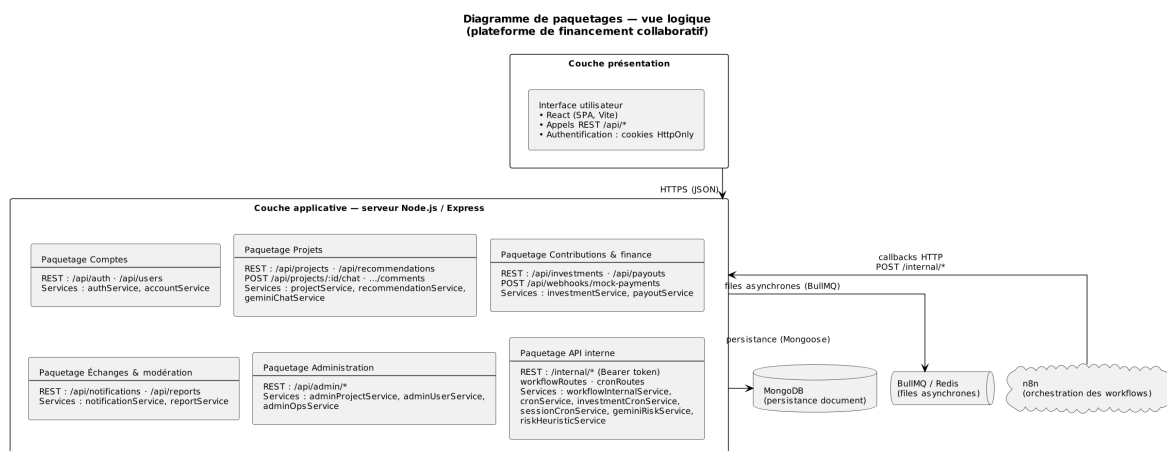


FIGURE 3.16 – Diagramme de paquetages de l'application
(diagramme_de_paquetage/diagramme_paquetage.png)

3.4.5 Décisions de conception structurantes

Les décisions suivantes ont été prises pour satisfaire les exigences non fonctionnelles (sécurité, résilience, traçabilité) :

- **Authentification par JWT (cookies HttpOnly)** : access token court + refresh token à plus longue durée, tous deux servis en cookies HttpOnly; Secure; SameSite, afin d'éviter l'exposition aux scripts client et au CSRF.
- **Base de données NoSQL document** : MongoDB choisi pour la flexibilité du document Project (champs variables, sous-document aiAnalysis) et la simplicité d'évolution du schéma.
- **Analyse IA orchestrée** : déclenchée par n8n, avec timeouts, tentatives et auto-rejet contrôlé (gap $\geq 30\%$).
- **Paiement asynchrone idempotent** : la confirmation arrive par *webhook* signé; la combinaison (provider, providerPaymentId) est unique pour absorber les tentatives du fournisseur.
- **Mise à jour conditionnelle atomique** du financement : updateOne sur projects avec contrainte currentFunding + amount $\leq$ fundingGoal pour empêcher tout sur-financement, même en concurrence.
- **Files BullMQ + DLQ** : tentatives exponentielles, stockage des échecs définitifs dans des collections dédiées pour reprise manuelle par l'administrateur.
- **Chiffrement des coordonnées bancaires** : algorithme AES-256-GCM avec IV par enregistrement et clé maître chargée hors code, pour respecter les exigences de confidentialité

des RIB.

- **Limitation de débit (*rate limiting*)** : en sortie API publique, plus stricte sur les surfaces sensibles (commentaires, signalements, authentification) pour limiter les abus.

3.5 Modèle de données MongoDB

3.5.1 Collections et structure des documents

Le modèle de données est dérivé du modèle du domaine et des schémas Mongoose. Comme la solution repose sur une base **NoSQL** (MongoDB), la structure est décrite sous forme de **collections** et de **références** (*ObjectId*) plutôt que sous forme de tables relationnelles. Dans ce rapport, la présentation est fournie **sans figures** afin d'éviter toute dépendance à des exports/outils externes.

- **users** : stocke les identifiants, le rôle, le statut actif, les jetons de réinitialisation, et le profil (préférences de risque, catégories).
- **projects** : contient les détails du projet ainsi que l'analyse IA embarquée (**aiAnalysis**).
- **investments** : lie un investisseur à un projet avec un montant et un statut.
- **transactions** : enregistre les informations de paiement (démon / extensible).
- **payouts** : stocke les retraits (statut, coordonnées bancaires chiffrées, suivi du virement).
- **comments** : stocke les commentaires (modération, masquage, suppression).
- **reports** : gère les signalements.
- **notifications** : stocke les notifications in-app.
- **auditLogs** : journalise les actions sensibles (traçabilité).
- **failedWorkflowEvents** : stocke les échecs des workflows pour reprise manuelle.
- **failedCancellationEvents** : stocke les échecs d'annulation de paiement lors de la désactivation d'un utilisateur.
- **failedRefundEvents** : stocke les échecs de remboursement (sur-financement, expiration, etc.).
- **failedPayoutEvents** : stocke les échecs de traitement des retraits.

3.5.2 Justification du modèle NoSQL

Le choix de MongoDB repose sur la nature semi-structurée des données. Les projets contiennent des champs variables (images, documents, analyse IA) qui évoluent au fil du temps.

Un modèle relationnel imposerait des tables supplémentaires ou des colonnes redondantes, tandis que MongoDB permet d'ajouter des champs sans migration. De plus, la possibilité d'utiliser des sous-documents (embedding) pour `profile` et `aiAnalysis` améliore les performances de lecture.

3.5.3 Indexation et relations

Pour optimiser les performances, les index suivants sont créés :

- `users.email` : unique.
- `projects.status` et `projects.createdAt` : pour filtrer les projets actifs.
- `projects.category` : pour les recommandations.
- `investments.investorId` et `investments.projectId` : pour retrouver rapidement les investissements d'un utilisateur ou d'un projet.
- `transactions.investmentId` : index simple (plusieurs transactions peuvent exister pour un même investissement, par exemple en cas de tentative).
- `transactions.provider` et `transactions.providerPaymentId` : index unique pour assurer l'idempotence des webhooks.
- `notifications.userId` et `notifications.read` : pour afficher les notifications non lues.
- `payouts.projectId`, `payouts.creatorId` et `payouts.status` : suivis administratifs et file d'approbation.
- `comments.projectId`, `comments.userId`, `comments.isHidden` et `comments.deletedAt` : filtrage, modération et affichage public.
- `auditLogs.timestamp` et `auditLogs.action` : historique des actions sensibles.
- `failedRefundEvents.resolved`, `failedPayoutEvents.resolved` (et dates associées) : files d'échec exploitées par l'administration.
- `failedWorkflowEvents.resolved` et `failedWorkflowEvents.createdAt` : pour lister les échecs non résolus.
- `failedCancellationEvents.resolved` et `failedCancellationEvents.createdAt` : idem.

Les relations sont gérées par références (`ObjectId`). L'embedding est utilisé pour les profils utilisateur et l'analyse IA dans les projets. En MongoDB, les jointures sont effectuées au niveau de l'application en utilisant les méthodes `populate` (avec Mongoose) ou des requêtes manuelles.

3.5.4 Dictionnaire des données

Le tableau 3.1 détaille les principaux champs retenus au sein des collections décrites.

TABLE 3.1 – Dictionnaire des données

Collection	Champ	Description
users	email	Adresse email de l'utilisateur (unique).
	passwordHash	Mot de passe haché avec bcrypt.
	role	Rôle de l'utilisateur (USER ou ADMIN).
	isActive	Statut du compte (actif / désactivé).
	resetToken	Jeton de réinitialisation du mot de passe.
	resetTokenExpiry	Date d'expiration du jeton.
	profile.firstName	Prénom.
	profile.lastName	Nom de famille.
	profile.riskPreference	Tolérance au risque (LOW, MEDIUM, HIGH).
	profile.preferredCategories	Catégories de projets préférées.
projects	title	Titre du projet.
	description	Description détaillée.
	category	Catégorie (ex : Technologie, Art, etc.).
	fundingGoal	Montant cible.
	currentFunding	Montant collecté (mise à jour atomique conditionnelle).
	status	État du projet (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED).
	aiStatus	État de l'analyse IA (PENDING, COMPLETED, FAILED).
	aiAnalysis.report	Rapport structuré (résumé, avantages, inconvénients, améliorations, questions, score).
	aiAnalysis.riskLevel	Niveau de risque (LOW, MEDIUM, HIGH).
investments	investorId	Référence vers l'utilisateur investisseur.
	projectId	Référence vers le projet.
	amount	Montant investi (strictement positif).
	status	État de l'investissement (INITIATED, SUCCESS, FAILED, CANCELLING, CANCELLED, REFUNDED).

Collection	Champ	Description
	updatedAt	Date de dernière mise à jour (utilisée pour repérer les contributions restées en INITIATED trop longtemps).
transactions	investmentId	Référence vers l'investissement.
	provider	Fournisseur de paiement (mock en démonstration).
	providerPaymentId	Identifiant du paiement chez le fournisseur (index unique avec provider pour l'idempotence).
	status	État de la transaction (PENDING, SUCCEEDED, FAILED, REFUNDED, CANCELLED).
payouts	projectId	Projet concerné ; contrainte d'unicité (au plus un retrait par projet).
	creatorId	Porteur de projet bénéficiaire.
	amount	Montant du versement demandé ou approuvé.
	status	Cycle de vie (PENDING, READY, PROCESSING, COMPLETED, CANCELLED, FAILED).
	bankDetails	Coordonnées bancaires chiffrées (AES-256-GCM) lorsque saisies.
	provider / providerTransfer Id	Fournisseur de virement et identifiant côté prestataire (démonstration / extensible).
	failureReason / horodatages (bankDetailsProvidedAt, etc.)	Suivi des échecs et jalons métier du virement.
comments	projectId / userId	Projet commenté et auteur du message.
	content	Texte du commentaire (longueur plafonnée côté schéma).
	isHidden / hiddenReason / hiddenBy / hiddenAt	Modération administrative : masquage et traçabilité.
	deletedAt / deletedBy / deletedByRole	Suppression (« soft delete ») par l'auteur ou l'administrateur.
reports	reporterId	Référence vers l'utilisateur ayant signalé.
	projectId	Référence vers le projet signalé.
	type	Type de signalement (FRAUD, SPAM, etc.).
	status	État du signalement (PENDING, RESOLVED).
notifications	userId	Référence vers l'utilisateur destinataire.
	type	Type de notification (ex. PROJECT_APPROVED).
	read	Indicateur de lecture (booléen read).

Collection	Champ	Description
auditLogs	actorId / actorRole	Auteur de l'action (utilisateur référencé) et rôle (USER, ADMIN).
	action / targetType / targetId	Verbe métier (EXPIRE_PROJECT, etc.) et cible typée avec identifiant.
	details / timestamp	Charge utile libre (Mixed) pour le contexte, date de l'événement.
failedRefundEvents	investmentId / projectId	Investissement et projet concernés par l'échec de remboursement.
	reason	Motif (OVERFUNDING, EXPIRY, désactivation utilisateur, etc.).
	error / retryCount / resolved	Diagnostic, tentatives et indicateur de résolution manuelle.
	resolvedBy / resolvedAt	Traçabilité de la clôture côté administration.
failedPayoutEvents	payoutId	Retrait ayant échoué définitivement côté file / worker.
	error / retryCount / resolved	Diagnostic, tentatives et reprise administrative.
	resolvedBy / resolvedAt	Audit de résolution lorsque traité par un administrateur.
failedWorkflowEvents	workflowType	Type de workflow (risk-analysis, notification).
	payload	Données initiales du workflow.
	error	Message d'erreur.
	retryCount	Nombre de tentatives déjà effectuées.
	resolved	Indique si l'échec a été résolu (manuellement).
failedCancellationEvents	investmentId	Investissement concerné.
	error	Message d'erreur.
	retryCount	Nombre de tentatives.
	resolved	Indique si l'échec a été traité.

3.6 Conclusion

Ce chapitre a opéré la transposition des besoins métier dans une conception UML homogène : les diagrammes de séquence en explicitent les enchaînements pour les cas critiques, le diagramme de classes en restitue la structure conceptuelle, et l'architecture en couches en situe les composants asynchrones (n8n, BullMQ/Redis) ainsi que les arbitrages structurants (JWT dans des cookies HttpOnly, idempotence des *webhooks*, atomicité des mises à jour de financement, chiffrement AES-256-GCM des RIB, recours à des files DLQ pour la reprise des échecs). Le **modèle**

de données MongoDB et le dictionnaire des données constituent le référentiel directement mobilisable par l'implémentation exposée au chapitre suivant.

Chapitre 4

Implémentation et tests

4.1 Introduction

Ce chapitre documente la réalisation technique de la plateforme. Il expose en premier lieu l'environnement de réalisation (logiciel, outillage et modalités de collaboration), puis décline les choix technologiques, illustre la mise en œuvre des cas d'utilisation les plus structurants (authentification, projets, paiement simulé, recommandations, notifications), présente la stratégie de validation et esquisse le schéma de déploiement. Les captures d'écran fournies reflètent les libellés définitifs adoptés dans l'application ; les diagrammes de séquence d'analyse du chapitre 3 ont servi de référence directe durant l'implémentation.

4.2 Environnement de réalisation

4.2.1 Environnement logiciel

Les principaux outils et bibliothèques utilisés sont présentés ci-dessous selon le canevas suivant : **rôle général**, **usage dans l'application** livrée, puis **atouts** retenus pour le développement. Les icônes reproduites proviennent des chartes graphiques des éditeurs concernés.

- **Backend**

- **Node.js (runtime) et pile serveur** : Node.js est le moteur d'exécution JavaScript côté serveur. **Express 5** structure l'API REST (/api/*, routes internes /internal/*, webhooks). **Mongoose 9** matérialise les collections et la validation au plus près des

documents ; **Zod 4** renforce le contrôle des entrées HTTP ; **BullMQ 5** pilote les jobs asynchrones ; **Nodemailer**, **jsonwebtoken** et **bcrypt** couvrent respectivement l'e-mail transactionnel, les jetons d'authentification et le hachage des mots de passe (versions principales indiquées dans `backend/package.json`).

Dans l'application : la logique métier est découpée en services (`*Service.js`) raccordés aux routeurs Express ; les opérations sensibles au temps (envois SMTP, relances) sont confiées aux workers plutôt qu'exécutées de manière bloquante dans le cycle requête-réponse.

Avantages : même langage que le frontend sur la partie « données et API », modèle non bloquant adapté aux entrées/sorties réseau et à la base de données, écosystème *npm* très fourni pour intégrer files, validation et sécurité.



- **Redis (files et état des jobs) :** Redis est une base clé-valeur en mémoire utilisée ici comme magasin de données pour les files d'attente.

Dans l'application : BullMQ s'appuie sur Redis (via **ioredis**) pour persister l'état des jobs (retardements, tentatives, verrous légers) avant toute remontée métier vers les collections d'échecs définitifs lorsque les reprises automatiques sont épuisées.

Avantages : latence très faible, primitives adaptées aux files et au scheduling, bonne tenue en charge lors des pics (notifications, relances administrateur).



— Frontend

- **React.js :** librairie JavaScript pour construire une interface utilisateur à base de **composants** réutilisables et d'un rendu réactif sur l'état.

Dans l'application : les vues plein écran et pages métier sont regroupées dans

`frontend/src/pages` (parcours investisseur, porteur de projet, administration, paiement démo, etc.) ; les éléments réutilisables (cartes projet, en-têtes, garde-fous d'accès) sont placés dans `frontend/src/components` et `frontend/src/components/ui`. La navigation repose sur **React Router** : enveloppe `BrowserRouter` dans `App.jsx`, déclaration des routes dans `routes/AppRoutes.jsx`. Les appels au backend passent par **Axios**, centralisé dans `frontend/src/api/client.js` (`withCredentials` pour les cookies d'authentification, intercepteur qui tente un rafraîchissement de session sur réponse 401 puis rejoue la requête initiale).

Avantages : forte modularité par composants, mise à jour efficace du DOM via le DOM virtuel, écosystème mature (hooks, outils de débogage, bibliothèques complémentaires).



- **Vite** : outil de build et serveur de développement orienté modules ES, chargé d'assembler le frontend pour le navigateur.

Dans l'application : le point d'entrée `main.jsx` monte l'arbre React sur le nœud `#root` ; la configuration (`vite.config.*`) fixe le serveur de développement, les alias de chemins et la cible de production. Le rechargement à chaud accélère les itérations sur les pages et composants.

Avantages : démarrage rapide du serveur de dev, feedback quasi instantané lors des modifications, configuration sobre comparée à des chaînes Webpack historiques plus lourdes.



- **Bootstrap 5** : framework CSS et composants JavaScript (modales, dépliants) pour uniformiser la mise en page responsive.

Dans l'application : les feuilles et scripts Bootstrap sont importés dans `main.jsx` aux côtés des styles propres à la plateforme (`index.css`, `styles/platform.css`), ce qui fournit la grille, les utilitaires d'espacement et les composants de base réemployés dans les pages.

Avantages : cohérence visuelle sur mobile et bureau, gain de temps sur les patterns UI courants (formulaires, grilles, navigation), documentation extensive.



- **MongoDB :** base de données **NoSQL** orientée documents, adaptée aux schémas évolutifs et aux regroupements logiques (sous-documents, tableaux embarqués).

Dans l'application : les collections `users`, `projects`, `investments`, `transactions`, `payouts`, `notifications`, etc., sont décrites avec Mongoose; le document `Project` embarque notamment `aiAnalysis`. Les transactions multi-documents (replica set requis en production pour cette fonctionnalité) sécurisent les opérations qui exigent une atomicité (ex. mise à jour du financement avec contrainte de plafond).

Avantages : flexibilité du modèle document, performances de lecture lorsque les données affichées ensemble sont stockées ensemble, alignement naturel avec les objets manipulés côté Node.js.



— **Orchestration et exécution conteneurisée**

- **n8n :** plateforme d'orchestration visuelle de workflows (nœuds HTTP, webhooks, branchements conditionnels, politiques de nouvelle tentative).

Dans l'application : un workflow n8n porte l'analyse de risque à la soumission d'un projet : réception du déclenchement, appel au service LLM, normalisation de la réponse, rappel des routes `/internal/*` du backend pour persister le résultat ou

tracer l'échec. Le chatbot projet (/api/projects/:id/chat) reste géré directement par Express dans la version livrée.

Avantages : découverte visuelle des enchaînements, évolutions d'intégration sans redéployer tout le cœur métier, mécanismes de *retry* configurables.



- **Docker** : technologie de conteneurisation qui encapsule une application et ses dépendances dans une image reproductible.

Dans l'application : l'instance n8n utilisée pour concevoir et valider le workflow d'analyse a été exécutée dans un conteneur dédié, isolée de la pile Node/React locale (voir aussi la sous-section « Conteneurisation (Docker) »).

Avantages : parité d'environnement entre développeurs et rapprochement d'un déploiement « image + orchestrateur », isolation des ports et des versions sans pollution de la machine hôte.



— Outillage

- **Visual Studio Code** : éditeur de code extensible, utilisé comme environnement de développement intégré quotidien.

Dans l'application : édition conjointe du backend Node/Express, du frontend React/Vite, des scripts E2E et des fichiers de configuration ; coloration et navigation dans les schémas Mongoose, les routeurs et les composants JSX.

Avantages : intégration Git, terminal et débogueur dans une seule fenêtre, marketplace d'extensions (ESLint, Prettier, clients REST légers), prise en charge native de JavaScript et JSX.



- **Git / GitHub** : Git assure le suivi de version ; GitHub héberge le dépôt distant partagé avec l'encadrant et l'architecte d'entreprise (voir § 4.2.3).

Dans l'application : historique des fonctionnalités par branches et *commits*, revue des changements avant intégration, diffusion des livrables et des captures associées au rapport.

Avantages : traçabilité fine, sauvegarde centralisée, facilitation du travail collaboratif et des relectures.



- **Enterprise Architect 15** : outil de modélisation UML et d'ingénierie dirigée par les modèles.

Dans l'application : production des diagrammes de cas d'utilisation, de séquence, de classes (y compris classes participantes) et de paquets exportés dans ce rapport, en les recoupant avec le code livré.

Avantages : formalisme UML reconnu, traçabilité des exigences vers la conception, supports graphiques exploitables en soutenance.



- **Postman** : client HTTP pour composer, rejouer et documenter des requêtes vers l'API.

Dans l'application : tests manuels des routes `/api/*` (authentification, projets, investissements, webhooks simulés) et validation des corps JSON / codes de statut

pendant le développement.

Avantages : reproductibilité des scénarios, collections partageables, inspection rapide des en-têtes et cookies.



- **Tests automatisés et validation manuelle :** les scénarios de bout en bout s'appuient sur le **même runtime Node.js** que le backend.

Dans l'application : les scripts `e2e-lifecycle-payout-api.js`, `e2e-lifecycle-scenarios.js`, `e2e-edgecases-api.js` et `e2e-interactions.js` enchaînent création de données, transitions de statut, paiements simulés et cas d'erreur ; ils complètent les tests manuels sur l'interface React.

Avantages : détection précoce des régressions sur les parcours critiques, exécution sans navigateur pour la couche API, automatisation rejouable avant chaque livraison.



4.2.2 Conteneurisation (Docker) et expérimentation n8n

Les workflows d'analyse de risque (réception d'un déclencheur, appel au service LLM, normalisation de la réponse, rappel des routes internes du backend) ont été mis au point et testés dans un **environnement Docker**. L'instance **n8n** y est exécutée dans un conteneur dédié, ce qui permet : (i) d'**isoler** l'orchestrateur du reste de la machine de développement ; (ii) d'obtenir un déploiement **reproductible** (même image, mêmes ports, même version) entre séances de travail ; (iii) de se rapprocher d'une **cible d'hébergement** où les services sont souvent packagés de la même manière. En pratique, cette base a servi à valider les nœuds du graphe (HTTP, Webhooks, gestion des erreurs, retries) avant de documenter le flux dans le rapport et de le corrélérer avec les appels `/internal/*` côté API.

4.2.3 Gestion de version et collaboration (GitHub)

Le dépôt de code de la plateforme (backend Node.js/Express, frontend React, scripts E2E, éventuelles configurations) est **poussé sur GitHub**. Le dépôt assure la **traçabilité** des évolutions (*commits*, branches, éventuelles *pull requests*) et constitue la référence unique pour les livrables du PFE. L'accès au dépôt a été **partagé** avec l'**encadrant académique** afin de permettre le suivi et la relecture du travail, ainsi qu'avec l'**architecte d'entreprise** de l'organisme d'accueil pour aligner les choix d'implémentation sur la vision d'architecture et faciliter les échanges techniques (revue de structure, conventions, points d'intégration).

4.2.4 Synthèse des choix technologiques

Les tableaux 4.1 et 4.2 synthétisent les technologies retenues et la justification de chaque choix (socle applicatif d'une part, IA/orchestration et exploitation d'autre part).

Composant	Technologie	Justification
Frontend	React + Vite + Bootstrap 5	Composants réutilisables, écosystème riche, UI cohérente responsive.
Backend	Node.js + Express	Environnement JavaScript unifié, non bloquant, adapté aux API REST et aux traitements asynchrones.
Validation	Zod	Schémas déclaratifs en JavaScript (zod), validation des entrées API et messages d'erreur exploitables côté contrôleur.
Base de données	MongoDB	Modèle document, sous-documents pour aiAnalysis, transactions multi-documents pour la cohérence.
Files asynchrones	BullMQ + Redis	Retentatives exponentielles, DLQ, supervision des jobs (e-mails, reprises).
Authentification	JWT (cookies HttpOnly)	Sans état, accès court / refresh long, protection contre l'XSS.
Chiffrement RIB	AES-256-GCM	Confidentialité des coordonnées bancaires, IV unique par enregistrement.

TABLE 4.1 – Technologies utilisées — socle applicatif, données et sécurité

Composant	Technologie	Justification
IA (analyse / chatbot)	Service LLM + n8n (analyse); Express (chatbot)	Analyse orchestrée, tentatives, auto-rejet sur incohérence budgétaire ($\geq 30\%$), heuristiques explicables; chatbot via <code>/api/projects/:id/chat</code> .
Orchestration	n8n	Découplage des workflows d'analyse; évolutions sans toucher au cœur Express pour ces flux.
Conteneur local	Docker (instance n8n)	Isolation de n8n, reproductibilité des tests de workflow, rapprochement d'un déploiement conteneurisé.
Versionnage	GitHub	Traçabilité du code; dépôt partagé avec l'encadrant et l'architecte d'entreprise pour suivi et revue.
Hébergement (cible)	Hébergement cloud managé (frontend, backend, base)	Déploiement continu, certificat SSL, sauvegardes managées.

TABLE 4.2 – Technologies utilisées — IA, orchestration et exploitation (suite)

4.3 Réalisation technique des cas d'utilisation

La présente section décrit l'implémentation des principales fonctionnalités; elle s'adosse aux diagrammes de séquence d'analyse pour hiérarchiser les interactions attendues entre composants techniques.

Illustrations d'interface utilisateur : les captures d'écran sont conservées dans le dépôt sous `rapport/images/screens/`, chemins exprimés relativement à la racine du répertoire `rapport/`. Chaque figure indique explicitement le fichier graphique employé; ces ressources peuvent être réemployées à l'identique dans la version finale du PDF comme dans le support de maintenance. Parmi les illustrations citées figurent notamment les écrans de vérification d'e-mail (`codeverifemail.png`, `pageverif.png`), la suite du formulaire de projet (`creationdeprojet2.png`) ainsi que l'étape OTP du parcours d'investissement (`codeOTPinvestir.png`).

4.3.1 Authentification et gestion des utilisateurs

Description fonctionnelle

L'authentification repose sur des **JWT stockés dans des cookies HttpOnly** : un jeton d'accès (*access token*) et un jeton de rafraîchissement (*refresh token*). À la connexion, le serveur vérifie les identifiants et dépose ces deux jetons dans des cookies (HttpOnly, SameSite=Lax, Secure en production). Le jeton de rafraîchissement permet de prolonger la session sans ressaisir le mot de passe, via l'endpoint de rafraîchissement.

Les mots de passe sont hachés avec `bcrypt`. Le schéma utilisateur inclut un champ `isActive` (booléen) qui est initialement `false` à l'inscription. Un e-mail de vérification est envoyé avec un **code OTP** (et un lien en secours). Le compte est activé après saisie du code (ou clic du lien). L'administrateur peut également désactiver ou réactiver un compte. Les routes protégées utilisent des middlewares d'authentification basés sur des cookies `HttpOnly` (access/refresh) et un contrôle de rôle pour l'administration. Sur le frontend, un intercepteur `Axios` surveille les réponses 401 (jeton expiré); il appelle alors `/api/auth/refresh` et rejoue la requête originale.

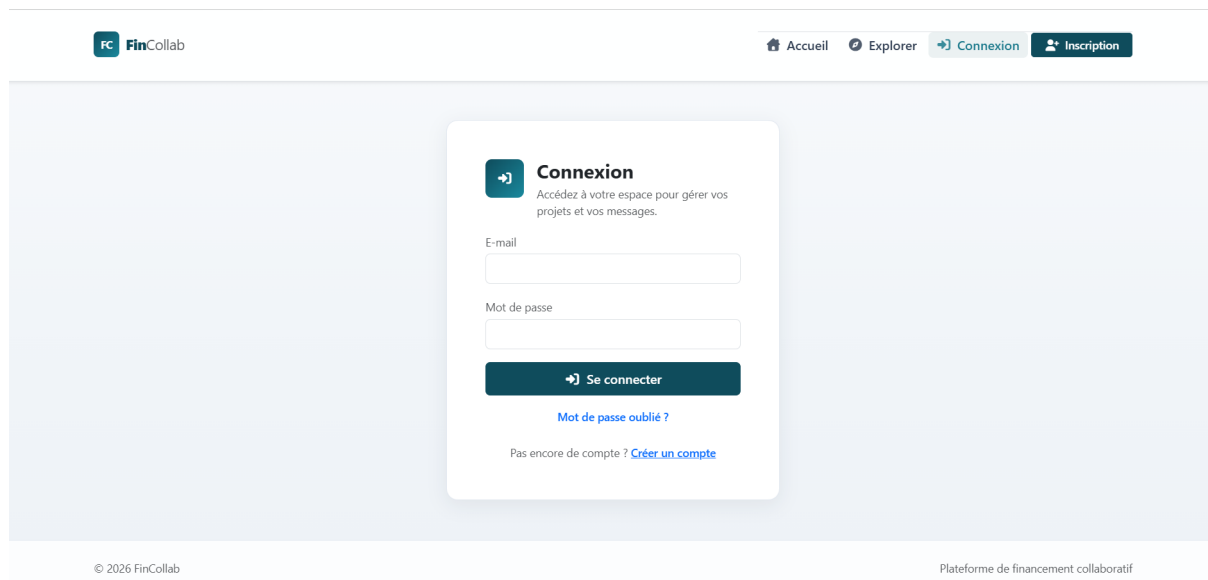
Bibliothèques utilisées

- `jsonwebtoken` — émission et vérification des jetons d'accès et de rafraîchissement.
- `bcrypt` — hachage des mots de passe (salt rounds ≥ 10).
- `cookie-parser` — lecture des cookies `HttpOnly` côté Express.
- `nodemailer` (combiné à `bullmq`) — envoi des e-mails OTP et de notifications de sécurité.
- `zod` — validation stricte des entrées (e-mail, mot de passe, OTP).
- `axios` (frontend) — appels API et intercepteur de rafraîchissement automatique.

Capture VSCode à insérer : intercepteur Axios de rafraîchissement

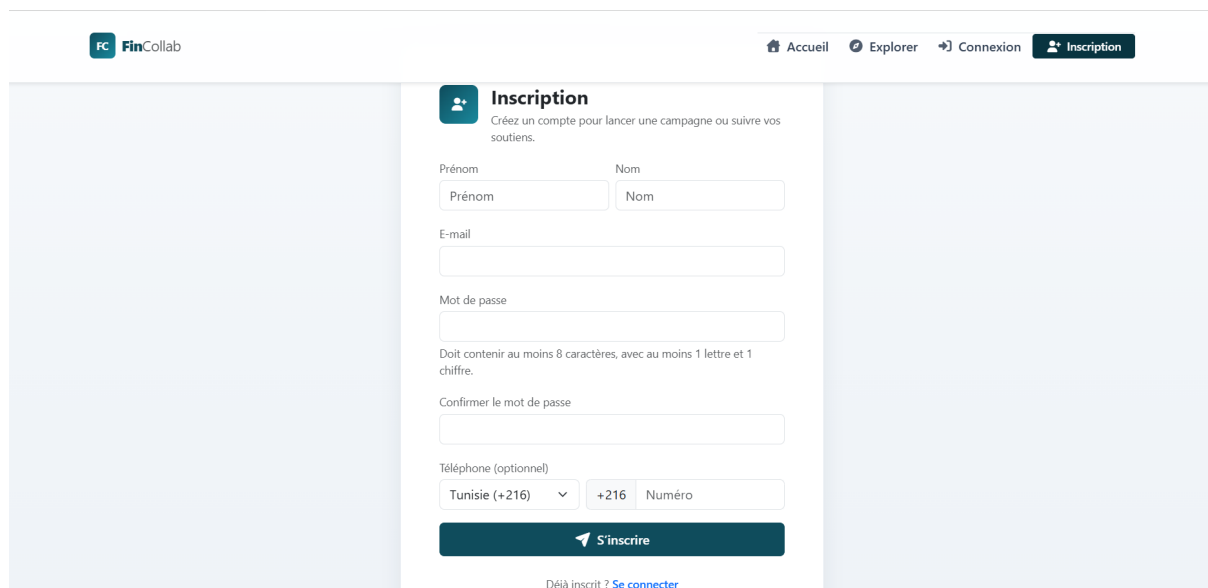
À faire : insérer une capture d'écran VSCode montrant l'intercepteur `Axios` qui gère une réponse 401 en appelant `/api/auth/refresh`, puis en rejouant la requête initiale.

Captures d'écran



The screenshot shows the login page of the FinCollab platform. The header includes the FinCollab logo and navigation links: Accueil, Explorer, Connexion, and Inscription. The main content area features a central login form titled "Connexion" with a sub-header "Accédez à votre espace pour gérer vos projets et vos messages." The form contains input fields for "E-mail" and "Mot de passe", a "Se connecter" button, a link for "Mot de passe oublié ?", and a link for "Créer un compte" for users who do not have an account. The footer displays the copyright notice "© 2026 FinCollab" and the platform name "Plateforme de financement collaboratif".

FIGURE 4.1 – Page de connexion (screens/login.png)



The screenshot shows the registration page of the FinCollab platform. The header includes the FinCollab logo and navigation links: Accueil, Explorer, Connexion, and Inscription. The main content area features a central registration form titled "Inscription" with a sub-header "Créez un compte pour lancer une campagne ou suivre vos soutiens." The form contains input fields for "Prénom", "Nom", "E-mail", "Mot de passe", and "Confirmer le mot de passe". It also includes a "Téléphone (optionnel)" section with a dropdown for "Tunisie (+216)" and a "Numéro" field. A "S'inscrire" button is at the bottom. A link for "Déjà inscrit ? Se connecter" is provided for existing users. The footer displays the copyright notice "© 2026 FinCollab" and the platform name "Plateforme de financement collaboratif".

FIGURE 4.2 – Formulaire d'inscription (screens/inscription.png)

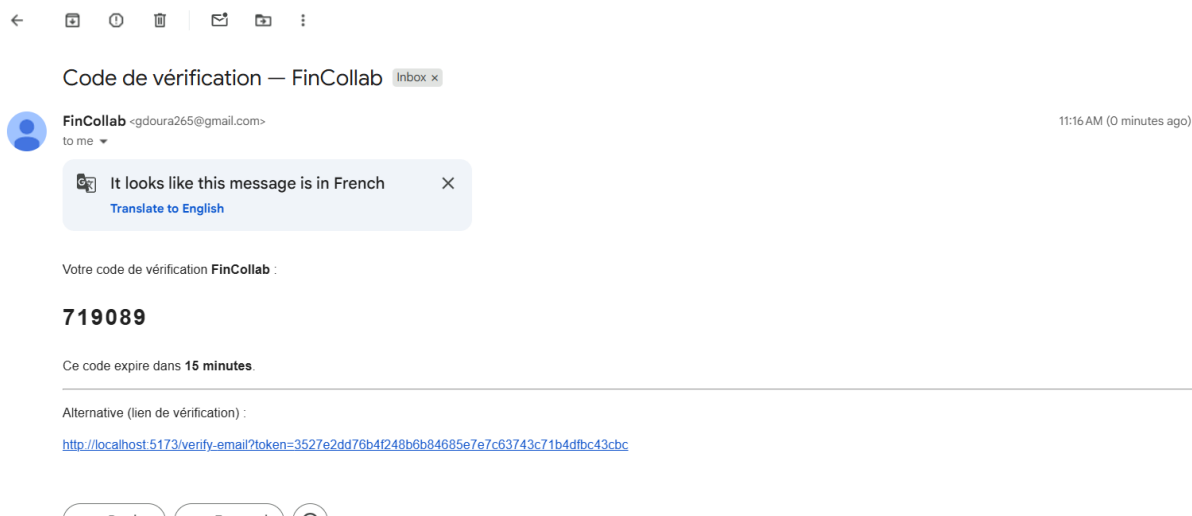


FIGURE 4.3 – Vérification d’e-mail : saisie du code (screens/codeverifemail.png)

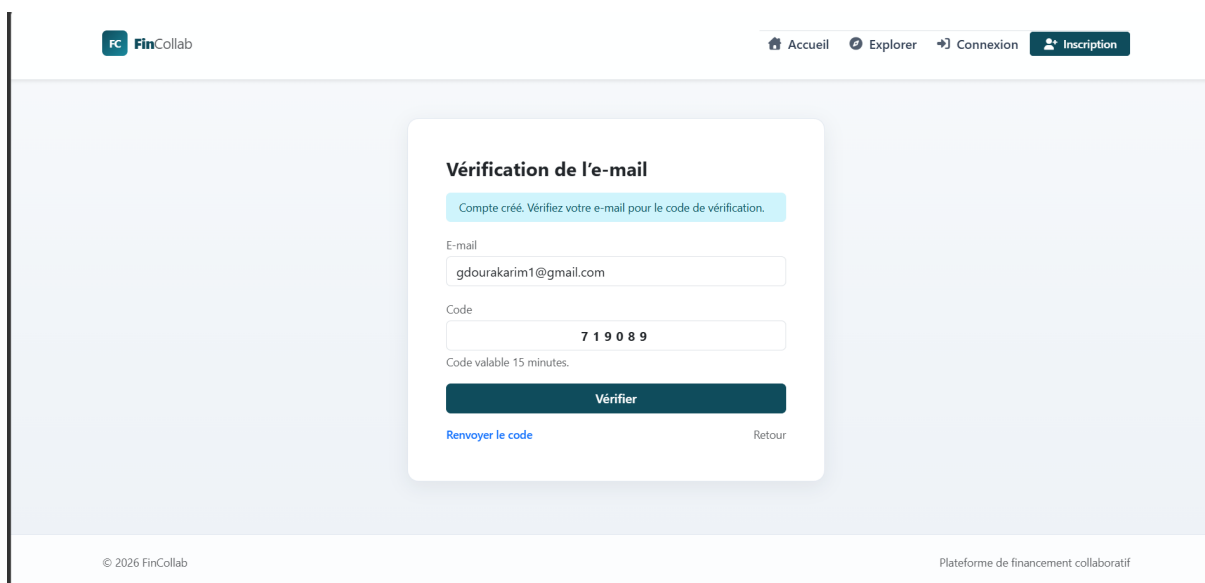


FIGURE 4.4 – Page de vérification / confirmation après validation (screens/pageverif.png)

Composants techniques associés

Cron de nettoyage des jetons de rafraîchissement expirés (cronService), table d’audit des tentatives de connexion, blocage progressif (*rate limiting* via *express-rate-limit*) sur `/api/auth/login` et `/api/auth/forgot-password`.

4.3.2 Gestion des projets

Description fonctionnelle

La création et la mise en ligne d'un projet suivent un cycle métier explicite :

1. Validation des données entrantes (champs obligatoires, type de données).
2. Création du projet en brouillon (DRAFT) par le porteur de projet.
3. Soumission explicite à l'analyse de risque : passage en **AWAITING_AI** et mise en file du workflow.

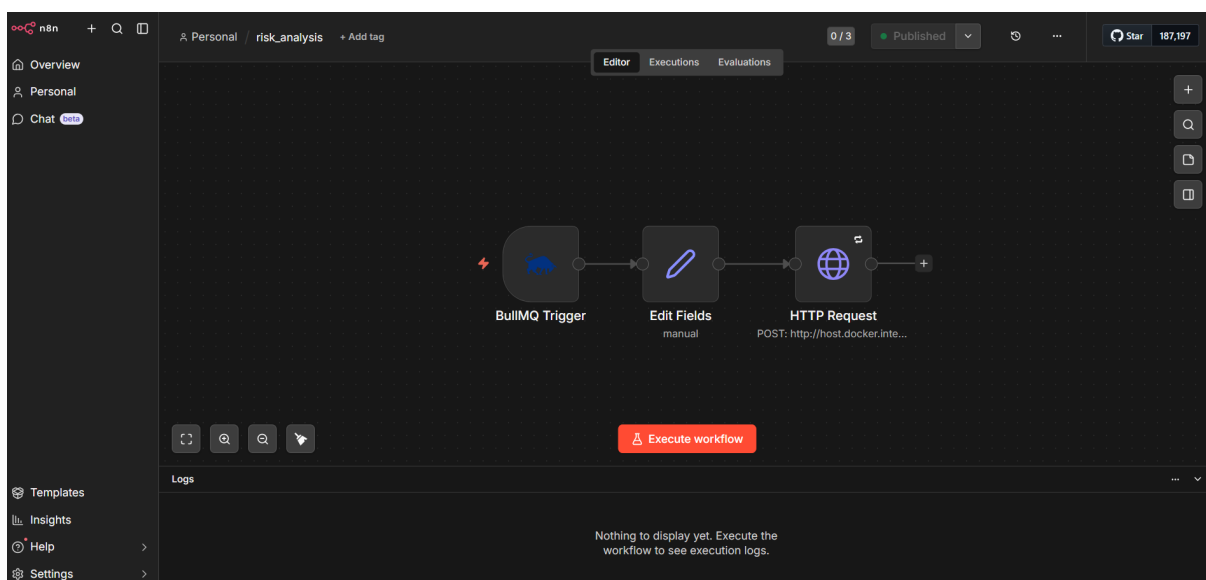


FIGURE 4.5 – Workflow n8n — analyse de risque (interface d'édition); fichier screens/n8nriskanalysis.png

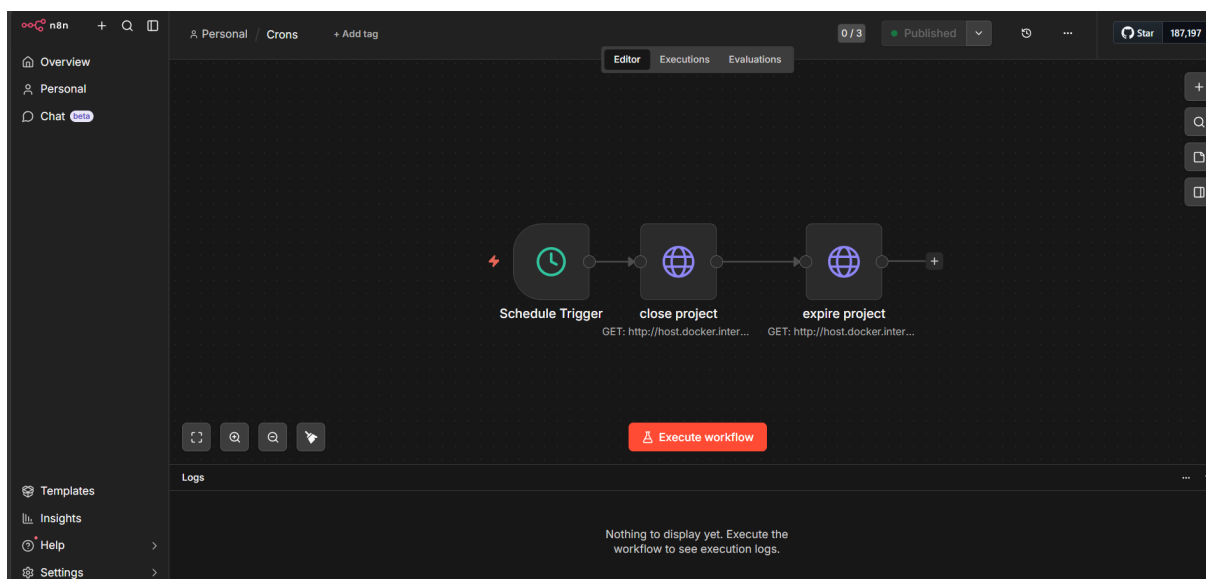


FIGURE 4.6 – Orchestration n8n complémentaire / scénarios planifiés ; fichier screens/n8ncrons .png

Le workflow n8n (voir figure 4.5) exécute les actions suivantes :

1. Appel au service LLM avec un prompt contenant les informations du projet.
2. Extraction du score de risque de la réponse.
3. Mise à jour du projet dans MongoDB avec le score et le niveau de risque, puis passage en UNDER_REVIEW (revue humaine).
4. Notification de l'administration (in-app + e-mail selon la politique).

En cas d'indisponibilité temporaire du service IA (quota/timeout/503), le projet reste en attente (AWAITING_AI) et le système planifie des tentatives automatiques. Un administrateur peut également déclencher une relance manuelle via l'interface d'administration (/api/admin/projects/:id/retry-ai).

Bibliothèques utilisées

- **mongoose** — modélisation des collections `projects` et `users`, transitions de statut, index secondaires.
- **zod** — validation des champs `title`, `description`, `category`, `fundingGoal`, `deadline`.
- **multer** — réception du média et de la pièce jointe du projet.
- **axios** (côté backend) — appel signé vers le `webhook` n8n d'analyse de risque.
- **n8n** — orchestration du workflow d'analyse IA + appel au service LLM.

Captures d'écran

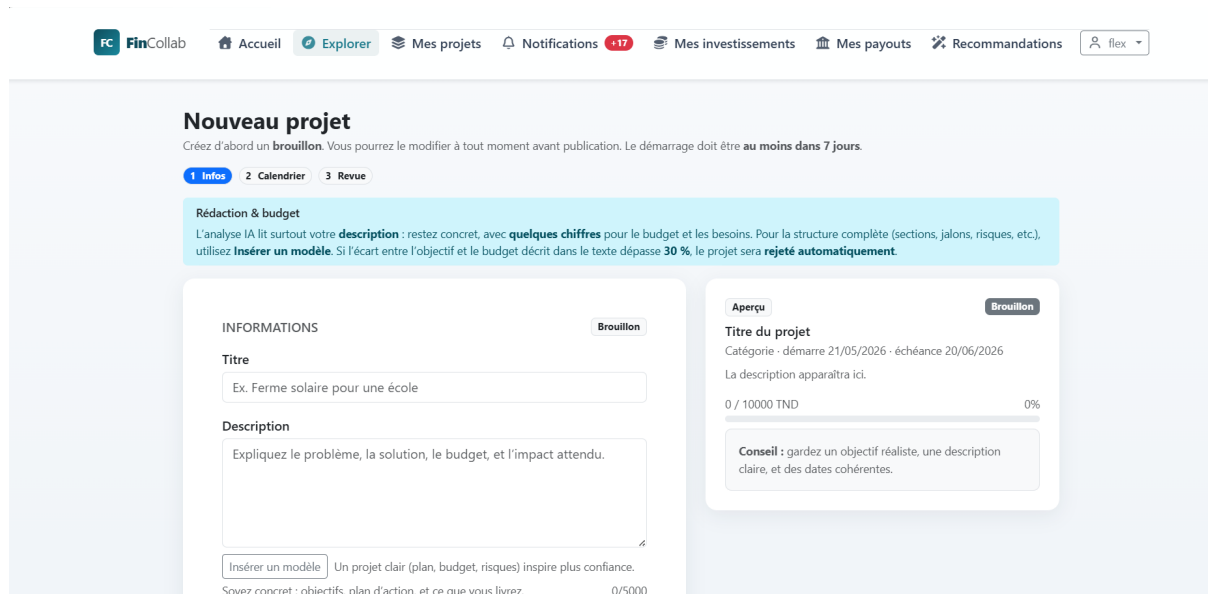


FIGURE 4.7 – Formulaire de création de projet — étape 1 (screens/creationdeprojet1.png)

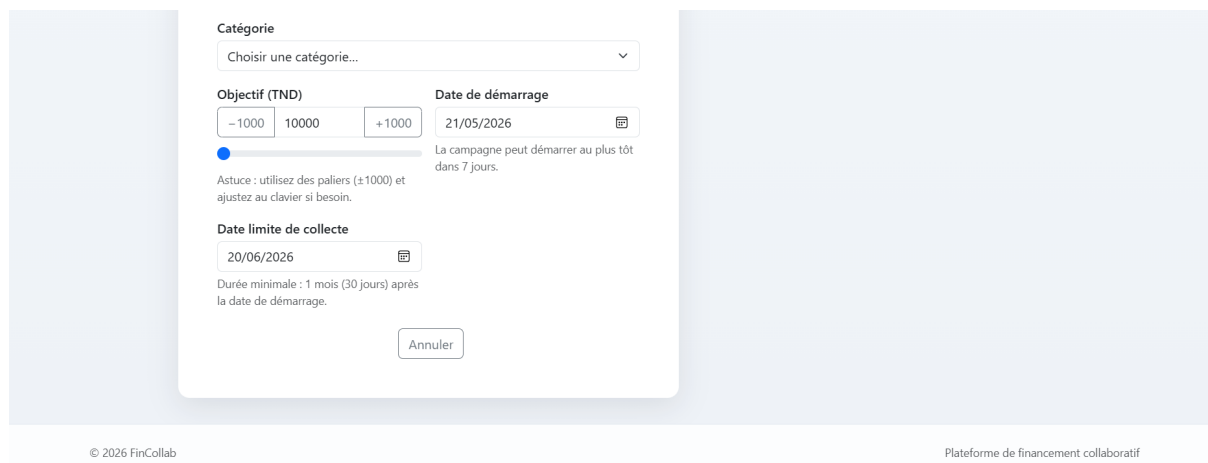


FIGURE 4.8 – Création de projet — étape complémentaire (screens/creationdeprojet2.png)

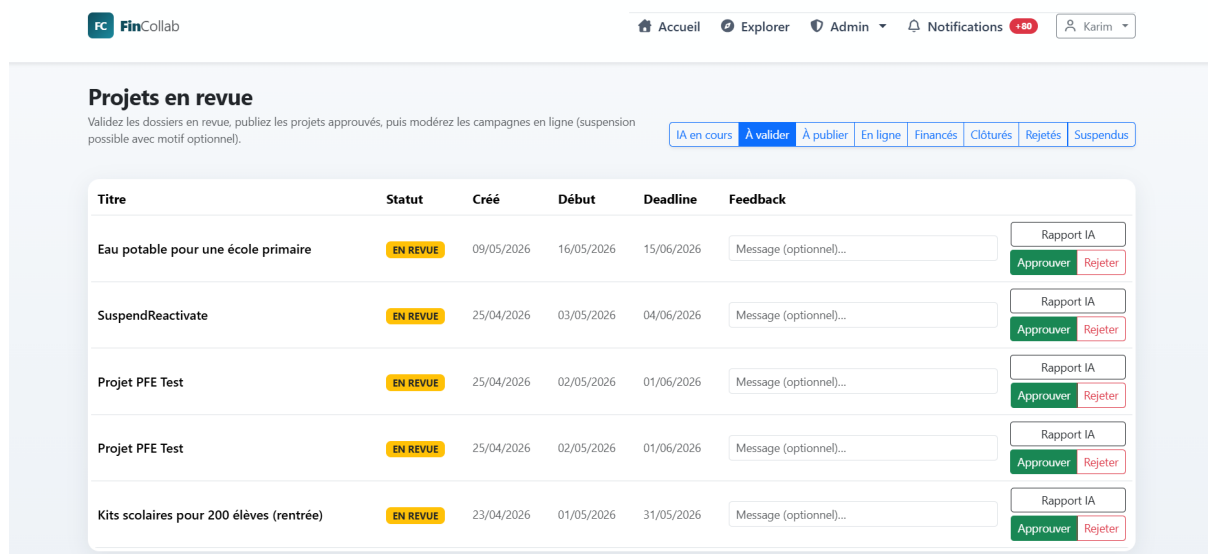


FIGURE 4.9 – Console d’administration : validation/rejet (screens/validerrejeter.png)

Composants techniques associés

Webhook signé HMAC entre le backend et n8n, file BullMQ pour les tentatives d’analyse IA, cron retryStuckAiAnalyses pour les projets bloqués en `AWAITING_AI`, `DLQ` `failedWorkflowEvents` pour les échecs définitifs, journal d’audit des transitions de statut.

4.3.3 Parcours de paiement (simulation)

Description fonctionnelle

Le parcours de paiement est simulé (mock) afin de valider la logique métier (statuts, webhooks signés, idempotence) sans dépendre d’un prestataire externe.

- **Création du lien de paiement** : lorsqu’un utilisateur initie un soutien, le backend retourne une URL de paiement simulé. L’investissement et la transaction sont créés en `INITIATED` / `PENDING`.
- **Webhook mock signé** : une confirmation HTTP signée met à jour transaction/investissement et ajoute le montant au `currentFunding` du projet, sous une condition atomique qui empêche de dépasser l’objectif de financement.
- **Coordonnées bancaires (RIB)** : les RIB des porteurs de projet sont chiffrés en AES-256-GCM (IV unique par enregistrement) avant stockage et utilisés au moment du *payout*.

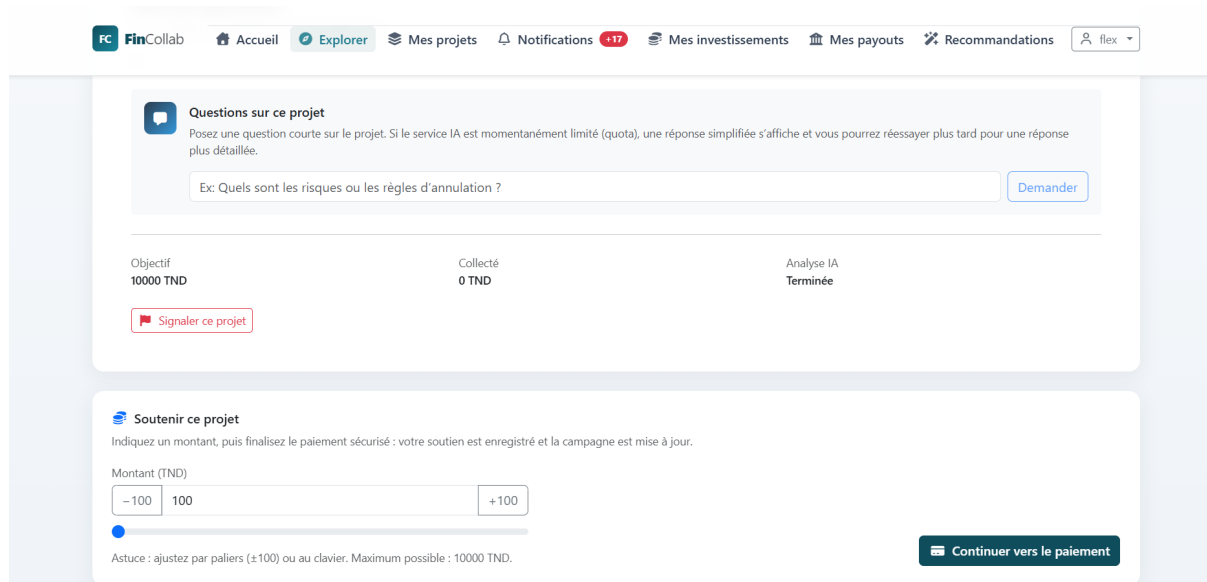
Bibliothèques utilisées

- mongoose — collections `investments`, `transactions`, `payouts`, transactions multi-documents (*replica set*).
- crypto (Node.js) — vérification HMAC du *webhook* et chiffrement AES-256-GCM des RIB.
- bullmq et ioredis — file de tentatives pour les remboursements et les paiements bloqués.
- zod — validation du *payload* de *webhook* et des saisies bancaires.

Capture VSCode à insérer : mise à jour atomique du financement

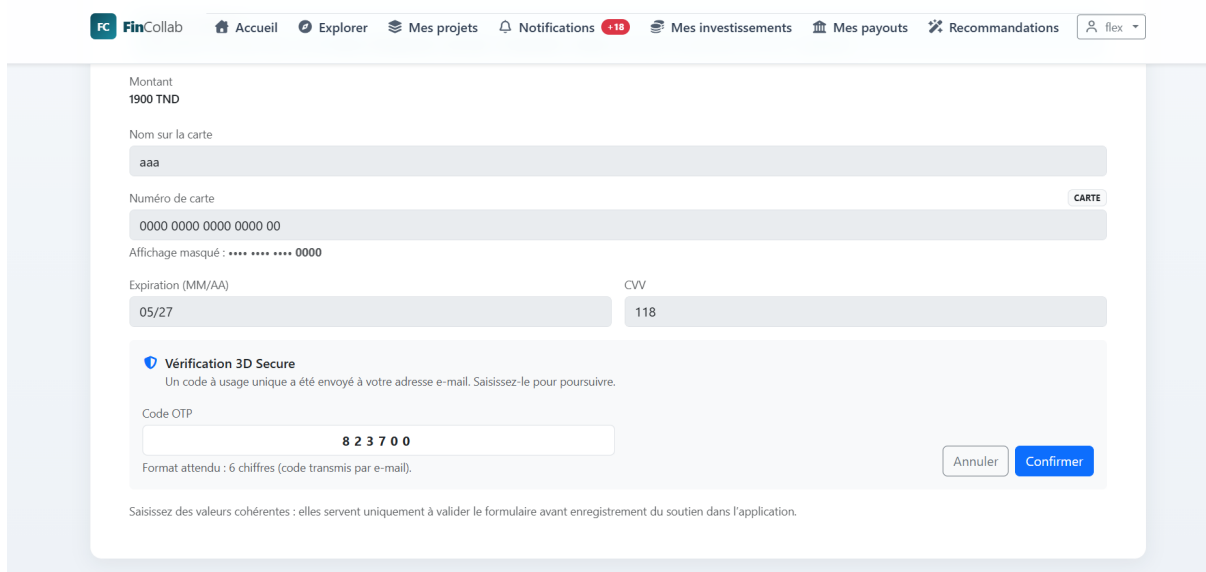
À faire : insérer une capture d'écran VSCode montrant la mise à jour atomique qui empêche `currentFunding` de dépasser `fundingGoal` lors d'un *webhook* concurrent. Cette approche garantit l'intégrité des données même en cas de *webhooks* concurrents.

Captures d'écran



The screenshot shows the 'FinCollab' web application interface. At the top is a navigation bar with links: Accueil, Explorer, Mes projets, Notifications (+17), Mes investissements, Mes payouts, Recommendations, and a user profile 'flex'. The main content area is divided into two sections. The first section, 'Questions sur ce projet', prompts the user to ask a question about the project, with a text input field containing the example 'Ex: Quels sont les risques ou les règles d'annulation ?' and a 'Demander' button. Below this is a summary table: 'Objectif 10000 TND', 'Collecté 0 TND', and 'Analyse IA Terminée', with a 'Signaler ce projet' button. The second section, 'Soutenir ce projet', instructs the user to indicate an amount for a secured payment. It features a 'Montant (TND)' input field with a range from -100 to +100, a progress bar, and a 'Continuer vers le paiement' button. A note at the bottom states: 'Astuce : ajustez par paliers (±100) ou au clavier. Maximum possible : 10000 TND.'

FIGURE 4.10 – Formulaire de soutien : investir (screens/investir.png)



Montant
1900 TND

Nom sur la carte
aaa

Numéro de carte
0000 0000 0000 0000 00

Affichage masqué : **** * 0000

Expiration (MM/AA)
05/27

CVV
118

Vérification 3D Secure
Un code à usage unique a été envoyé à votre adresse e-mail. Saisissez-le pour poursuivre.

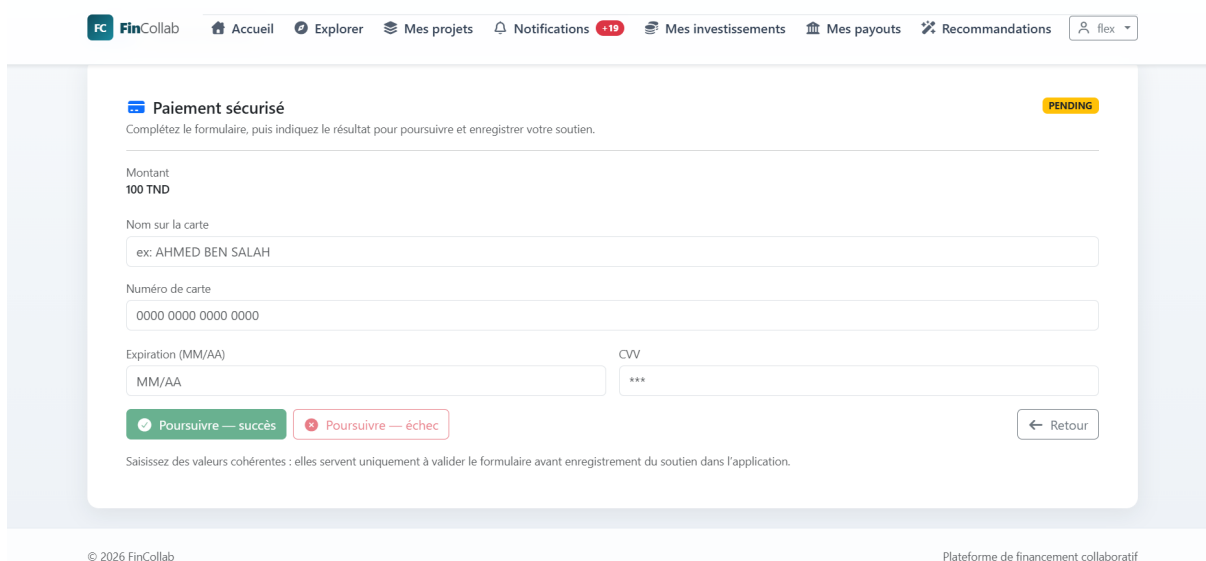
Code OTP
8 2 3 7 0 0

Format attendu : 6 chiffres (code transmis par e-mail).

Annuler Confirmer

Saisissez des valeurs cohérentes : elles servent uniquement à valider le formulaire avant enregistrement du soutien dans l'application.

FIGURE 4.11 – Étape de confirmation (code OTP / sécurisation du paiement démo) (screens/codeOTPinvestir.png)



Paieement sécurisé PENDING

Complétez le formulaire, puis indiquez le résultat pour poursuivre et enregistrer votre soutien.

Montant
100 TND

Nom sur la carte
ex: AHMED BEN SALAH

Numéro de carte
0000 0000 0000 0000

Expiration (MM/AA)
MM/AA

CVV

Poursuivre — succès Poursuivre — échec Retour

Saisissez des valeurs cohérentes : elles servent uniquement à valider le formulaire avant enregistrement du soutien dans l'application.

© 2026 FinCollab Plateforme de financement collaboratif

FIGURE 4.12 – Coordonnées bancaires (screens/cordonnebancaire.png); chiffrement AES-256-GCM en amont stockage

Composants techniques associés

Webhook HMAC idempotent (clé unique provider+providerPaymentId), file BullMQ refund-queue pour les remboursements, cron de retentative des *payouts* en échec, DLQ failedRefund et failedPayoutEvents pour reprise manuelle par l'administrateur.

4.3.4 Nettoyage des investissements abandonnés

Description fonctionnelle

Un job planifié (cron interne) exécute régulièrement un nettoyage des investissements restés au statut INITIATED au-delà d'un seuil (par défaut 30 minutes), en s'appuyant sur l'horodatage `updatedAt` (ou `createdAt`) de la collection `investments`. Ces investissements sont automatiquement marqués `FAILED`, la transaction associée est mise à jour, et une notification est envoyée à l'investisseur. Ce mécanisme évite l'accumulation d'enregistrements orphelins et améliore l'expérience utilisateur.

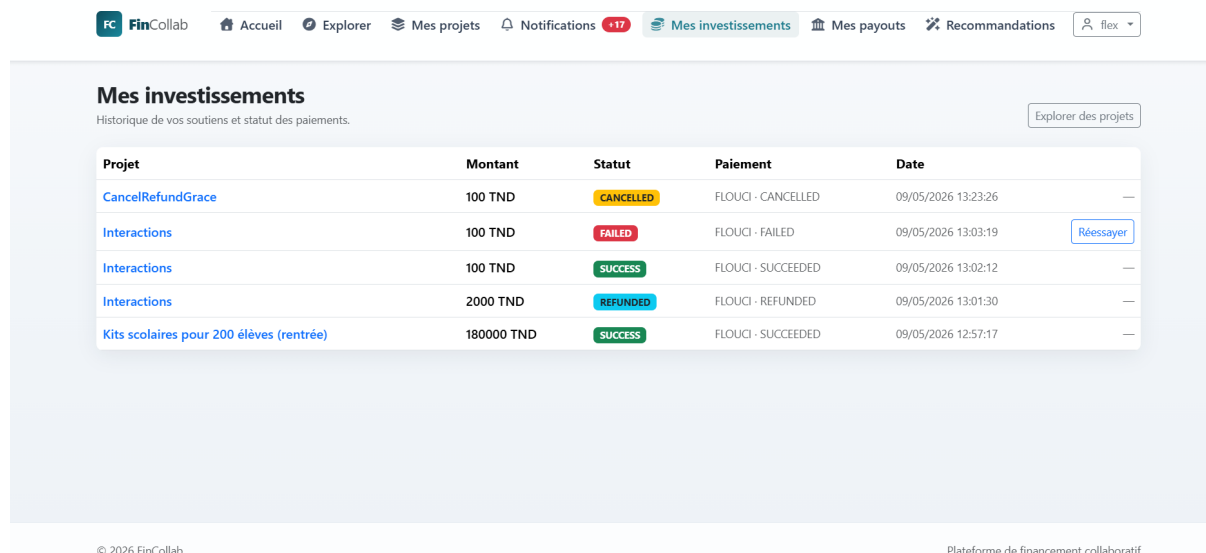
Bibliothèques utilisées

- `node-cron` — planification du job d'expiration.
- `mongoose` — requêtes ciblées sur `status`: `INITIATED` et ancienneté (`updatedAt/createdAt`).
- `bullmq` — production d'un événement de notification consommé en aval.

Capture VSCode à insérer : nettoyage périodique des investissements abandonnés

À faire : insérer une capture d'écran VSCode montrant la tâche cron qui identifie les investissements `INITIATED` trop anciens (via `updatedAt/createdAt`) et les marque en échec.

Capture d'écran



Projet	Montant	Statut	Paiement	Date
CancelRefundGrace	100 TND	CANCELLED	FLOUCI - CANCELLED	09/05/2026 13:23:26
Interactions	100 TND	FAILED	FLOUCI - FAILED	09/05/2026 13:03:19
Interactions	100 TND	SUCCESS	FLOUCI - SUCCEEDED	09/05/2026 13:02:12
Interactions	2000 TND	REFUNDED	FLOUCI - REFUNDED	09/05/2026 13:01:30
Kits scolaires pour 200 élèves (rentrée)	180000 TND	SUCCESS	FLOUCI - SUCCEEDED	09/05/2026 12:57:17

FIGURE 4.13 – Contributions / investissements (screens/mescontributions.png); statuts INITIATED, SUCCESS, FAILED

Composants techniques associés

Service interne cronService, file BullMQ pour la production des notifications associées, journalisation structurée des transitions de statut.

4.3.5 Moteur de recommandation

Description fonctionnelle

Le moteur est implémenté dans recommendationService.js (getRecommendationsForUser). Il est **entièrement déterministe** dans la version livrée : sélection des projets ACTIVE non archivés, calcul d'un score par projet à partir de quatre composantes (pondérations codées en dur) — adéquation aux catégories préférées du profil, adéquation au riskLevel issu de project.aiAnalysis, popularité (progression vers l'objectif) et récence de création — puis tri décroissant et découpe selon la limite demandée (défaut 8, plafonnée à 20). Aucun appel LLM n'est effectué dans ce service : l'IA intervient en amont lors de l'analyse de projet, dont le résultat est réutilisé ici.

Bibliothèques utilisées

— mongoose — requêtes find sur projects et users (lean()).

Capture VSCode à insérer : calcul du score de recommandation

À faire : insérer une capture d'écran VSCode montrant la fonction de scoring déterministe (catégories, riskLevel, progression, récence) et le tri/découpe.

Capture d'écran

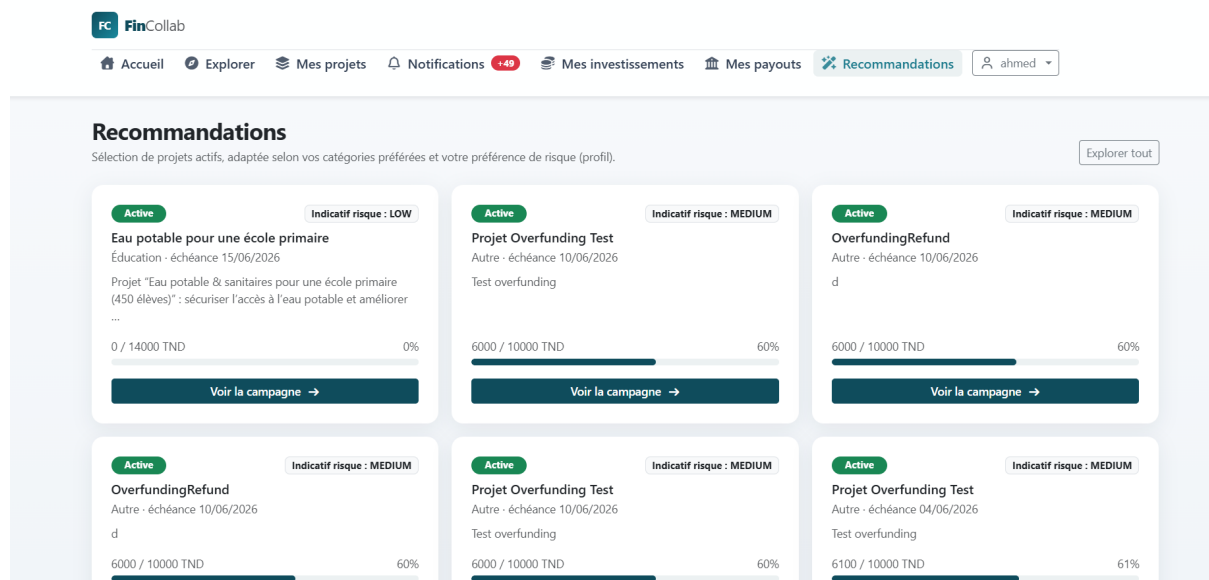


FIGURE 4.14 – Recommandations de projets (screens/recommendation.png)

Composants techniques associés

Cache mémoire de courte durée pour les recommandations par utilisateur ; le classement reste déterministe et traçable (profil utilisateur et champs project déjà en base).

4.3.6 Service de notifications

Description fonctionnelle

Les notifications *in-app* reposent sur la collection MongoDB `notifications` : chaque document est adressé à **un** utilisateur (`userId`), avec un booléen `read`, un type énuméré, titre, message et références optionnelles à une entité métier (`relatedEntityType` / `relatedEntityId`). L'API REST unique `/api/notifications` retourne **uniquement** les messages du compte authentifié ; elle accepte les paramètres de requête `page` et `limit` (plafonné à 30 éléments par page côté serveur), trie par `createdAt` décroissant (plus récent en premier), et fournit dans la réponse JSON le tableau `notifications`, le total `unreadCount` (pour le badge de la barre de navigation) et le booléen `hasMore` (obtenu en demandant une ligne supplémentaire côté base puis en tronquant à la taille de page). L'action PUT `/api/notifications/:id/read` marque comme lu si la notification appartient au demandeur. **Tous les rôles, y compris l'administrateur, utilisent ce même canevas** : il n'existe pas d'écran séparé listant les messages « de toute la plateforme » dans la version livrée. À côté de ce flux, les **échecs d'envoi SMTP** (jobs e-mail non délivrés) sont diagnostiqués via `/api/admin/failed-notifications` et une page d'administration dédiée (`/admin/email-failures`) permettant une relance ciblée — ce mécanisme est distinct des notifications in-app. Pour les événements critiques (sécurité, paiement, retrait, décision sur projet, etc.), un e-mail est produit selon la politique `emailNotificationPolicy`, via files BullMQ/Redis et workers ; les indisponibilités SMTP sont journalisées et peuvent donner lieu à reprises.

Bibliothèques utilisées

- `mongoose` — schéma `Notification`, index sur `userId` et champs de filtrage ; pas de champ `readAt` : le statut lu repose sur le booléen `read`.
- `bullmq` et `ioredis` — files `email-queue`, `notification-queue`, et files associées aux remboursements / *payouts*.
- `nodemailer` — envoi SMTP via le *worker* BullMQ.
- **n8n** — orchestration de l'analyse de risque ; le chatbot et la modération passent par l'API Express.

Capture VSCode à insérer : notification critique (in-app + e-mail)

À faire : insérer une capture d'écran VSCode montrant l'émission d'une notification in-app et l'ajout d'un job BullMQ pour l'e-mail (tentatives + backoff).

Capture d'écran

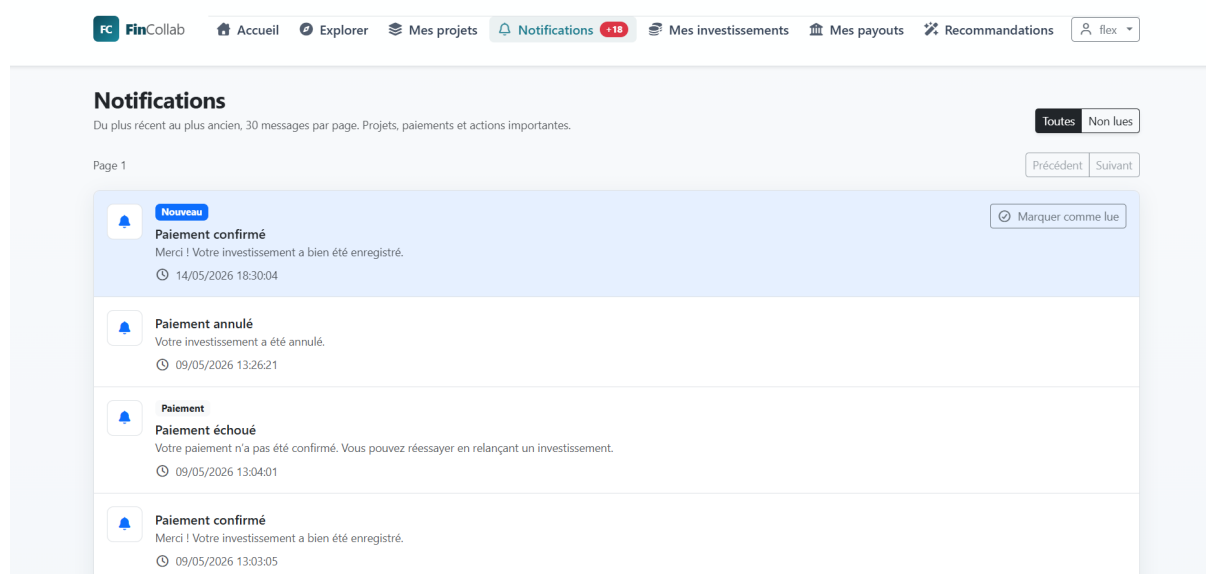


FIGURE 4.15 – Notifications (screens/notification.png) : liste personnelle paginée

Composants techniques associés

DLQ `failedWorkflowEvents` pour les événements `n8n` irrécupérables, collections d'événements d'échec côté e-mail et `payouts` exposées à l'administration pour reprise, politique de filtrage « critique ON / bruit OFF » pour les e-mails transactionnels.

4.3.7 Stratégie de test

La validation a été effectuée au moyen de **scripts E2E Node.js** qui simulent des parcours métier complets via l'API, en l'absence de *frameworks* de tests unitaires (le projet n'utilise ni Jest, ni Mocha, ni Cypress). Les fichiers concernés résident sous `backend/scripts/` :

- `e2e-lifecycle-payout-api.js` : inscription avec vérification OTP, création projet (DRAFT), soumission à l'analyse IA, validation et publication par l'administrateur, contri-

bution + *webhook* signé, création du *payout*, saisie des coordonnées bancaires chiffrées, approbation administrateur.

- `e2e-lifecycle-scenarios.js` et `e2e-edgecases-api.js` : scénarios limites (annulation d'investissement, relance de paiement, sur-financement et remboursement, transitions de statut invalides, erreurs de validation).
- `e2e-interactions.js` : interactions sociales (commentaires, signalements, chatbot).

Le script PowerShell `scripts/smoke.ps1` (racine du dépôt) enchaîne des requêtes GET anonymes (`/api/health`, `/api/projects/public`) pour une vérification rapide après déploiement ; le backend doit être démarré (URL par défaut `http://localhost:5000`, surchargée par `-BaseUrl` si besoin pour suivre la variable `PORT`).

En complément, des **tests manuels guidés** ont été réalisés sur l'interface web (parcours porteur de projet, investisseur, administrateur) afin de vérifier l'ergonomie, la cohérence des messages et la fidélité de l'affichage des statuts.

4.3.8 Déploiement

La plateforme est conçue pour être déployée sur un environnement cloud managé (ou sur un serveur institutionnel) avec séparation des composants :

- **Frontend** : application React servie via un hébergement web (HTTPS).
- **Backend** : service Node.js/Express exposant l'API REST.
- **Base de données** : MongoDB (replica set requis pour les transactions multi-documents).
- **Broker** : Redis pour les files BullMQ.
- **Orchestration** : instance n8n (auto-hébergée) pour les workflows.

Les exigences de sécurité (HTTPS, secrets gérés hors code, signatures *webhook*, chiffrement AES-256-GCM des RIB) sont identiques en local et en production ; seules les valeurs des secrets et l'origine CORS varient par environnement.

4.4 Conclusion

L'implémentation a permis de valider les principaux processus métier de la plateforme : cycle de vie complet des projets (`DRAFT` → `AWAITING_AI` → `UNDER_REVIEW` → `APPROVED` → `ACTIVE` → `FUNDED` / `CLOSED` / `SUSPENDED`, avec branche `REJECTED`), contribution avec paiement simulé idempotent, analyse de risque et recommandations, gestion résiliente des échecs (DLQ), boîte de notifications personnelle paginée (30 entrées, ordre antéchronologique) et administration (dont supervision

des envois SMTP en échec). La cohérence de bout en bout a été vérifiée par des scripts E2E Node.js et complétée par des tests manuels guidés sur l'interface. La conclusion générale qui suit met en perspective les apports, les limites et les pistes d'évolution.

Conclusion générale

Travaux réalisés et solutions apportées

Le travail réalisé converge vers une plateforme web de financement participatif couvrant les processus métier essentiels : création et gestion des campagnes, contribution des investisseurs, gouvernance par validation administrative, traitement des retraits, complétés par des dispositifs visant la transparence (rapport d'analyse de risque, notifications, modération). Les principales réalisations se déclinent comme suit :

- Un **parcours complet** porteur de projet : brouillon, soumission à l'analyse, revue/validation, publication, suivi de la collecte, puis préparation du retrait.
- Un **parcours complet** investisseur : exploration, contribution, suivi des investissements et gestion des incidents (paiement échoué, annulation, remboursement).
- Des **aides à la décision** : analyse de risque produisant un rapport structuré et recommandations adaptées au profil.
- Une **gouvernance et conformité** renforcées : rôles, modération, signalements, journalisation des actions, notifications personnelles de suivi et relances contrôlées des files d'échec (e-mails, *payouts*).

Sur le plan technique, plusieurs solutions structurantes ont été conçues pour traiter les difficultés rencontrées : un *webhook* de paiement **signé en HMAC et idempotent** grâce à un index unique (`provider`, `providerPaymentId`) ; une **mise à jour conditionnelle atomique** de la collection `projects` (`currentFunding` + montant $\leq$ `fundingGoal`) qui préserve l'intégrité du financement sous charge concurrente ; un **repli déterministe** pour le moteur de recommandation et un cron `retryStuckAiAnalyses` pour les projets bloqués en `AWAITING_AI` en cas d'indisponibilité du LLM ; quatre files *dead-letter* côté collections `failedWorkflowEvents`, `failedRefundEvents`, `failedPayoutEvents` et `failedCancellationEvents`, qui rendent visible toute erreur définitive dans les chaînes asynchrones `n8n` / `BullMQ` et permettent une reprise manuelle ; enfin, le **chiffrement AES-256-GCM** des RIB avec un IV unique par enregistrement

assure la confidentialité des coordonnées bancaires.

Indépendamment de la dimension strictement logicielle, le dispositif adopté illustre une démarche articulée d'**ingénierie des systèmes d'information**, associant capture des besoins, modélisation UML, conception des données et validation par scénarios métier.

Apports personnels et compétences acquises

Ce projet de fin d'études a constitué une expérience formatrice à plusieurs niveaux :

- **Développement full-stack** : conception d'une SPA React et d'API REST Express, avec séparation claire entre logique métier et accès aux données.
- **Sécurité applicative** : authentification par cookies HttpOnly, contrôle d'accès par rôles, *rate limiting*, signatures *webhook* et chiffrement applicatif.
- **Données et intégrité** : modélisation MongoDB, indexation, transactions multi-documents et gestion des cas critiques (sur-financement, remboursements, retraits).
- **IA appliquée au métier** : intégration d'un LLM via n8n pour l'analyse de risque des projets ; assistant conversationnel sur la fiche projet via l'API Express (`/api/projects/:id/chat`) et appel direct au LLM, avec gestion des quotas et des indisponibilités.
- **Analyse et modélisation** : formalisation des besoins, cas d'utilisation, diagrammes de séquence et de classes pour structurer le système d'information.
- **Validation** : conception de scénarios E2E bout en bout et contrôles de cohérence systématiques sur les statuts, transitions et messages utilisateurs.

La période consacrée à ce stage a permis d'opérer une mise en pratique systématique des enseignements de la licence tout en renforçant l'autonomie nécessaire à la conduite d'un projet d'envergure. Les échanges réguliers avec l'encadrant se sont révélés déterminants pour consolider les arbitrages techniques et pour lever les blocages ponctuels rencontrés. L'intégration de briques d'intelligence artificielle au sein d'un système d'information métier a en outre offert un terrain privilégié d'appréhender, sur le plan empirique, les exigences attachées à ces technologies (performance, exactitude, gestion des erreurs), au-delà du seul cadre conceptuel.

Critiques et limites

Malgré les résultats obtenus, certaines limites subsistent :

- Le moteur de recommandation est principalement fondé sur les préférences et des règles

de compatibilité (catégories/risque); l'absence de données historiques limite l'évaluation empirique.

- L'analyse de risque reste dépendante de la qualité de la description du projet; une revue humaine demeure nécessaire, surtout pour les projets sensibles.
- La plateforme n'inclut pas de version mobile native, bien que le site soit *responsive*.
- Les validations ont été réalisées par scénarios automatisés côté API et tests manuels; une automatisation IHM (tests UI) renforcerait encore la qualité.
- L'intégration d'un prestataire de paiement réel implique des exigences supplémentaires (KYC, *sandbox*, certificats, supervision) qui dépassent le périmètre PFE.

Perspectives

Plusieurs améliorations peuvent être envisagées pour enrichir le système :

- **Industrialiser le paiement** : intégrer un prestataire réel (*sandbox*), renforcer l'observabilité des *webhooks* et formaliser les contrôles de conformité.
- **Améliorer la décision** : affiner l'analyse de risque (meilleure explication, calibration) et enrichir les recommandations par apprentissage dès qu'un historique existe.
- **Pilotage SI** : ajouter des tableaux de bord (KPIs) orientés métier (taux de succès, délais de validation, volumes de contribution, incidents).
- **Qualité** : automatiser des tests UI et renforcer les tests de non-régression sur le cycle de vie projet/paiement/*payout*.

Ce travail illustre par ailleurs l'intérêt d'intégrer l'IA dans un système d'information métier afin d'améliorer la transparence (rapport lisible) et l'aide à la décision (recommandations), tout en conservant une gouvernance humaine (validation et modération); l'approche proposée peut être transposée à d'autres SI où la décision doit être **expliquée** et **traçable**.

Webographie

- Banque Centrale de Tunisie (BCT) — Cadre légal du financement participatif (Tunisie). <https://www.bct.gov.tn/>
- Object Management Group — UML 2.5.1 Specification. <https://www.omg.org/spec/UML/>
- OWASP — Top 10 Web Application Security Risks. <https://owasp.org/Top10/>
- React — Documentation officielle. <https://react.dev/>
- Node.js — Documentation officielle. <https://nodejs.org/>
- MongoDB — Documentation officielle. <https://www.mongodb.com/docs/>
- n8n — Documentation officielle. <https://n8n.io/>
- Scrum Guide — Guide officiel Scrum. <https://scrumguides.org/>

Bibliographie

- [1] Banque Centrale de Tunisie. (2020). *Loi n° 2020-37 du 2 juillet 2020 relative au financement participatif*. Journal Officiel de la République Tunisienne, n° 76.
- [2] Object Management Group. (2017). *Unified Modeling Language (UML) Specification, Version 2.5.1*. Consulté le 3 mai 2026. <https://www.omg.org/spec/UML/>
- [3] Belleflamme, P., Lambert, T., & Schwienbacher, A. (2014). Crowdfunding : Tapping the right crowd. *Journal of Business Venturing*, 29(5), 585-609.
- [4] Mollick, E. (2014). The dynamics of crowdfunding : An exploratory study. *Journal of Business Venturing*, 29(1), 1–16.
- [5] Ricci, F., Rokach, L., & Shapira, B. (2011). *Introduction to Recommender Systems Handbook*. Springer.
- [6] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3e éd.). Prentice Hall.
- [7] Zhang, J., & Liu, P. (2012). Rational herding in microloan markets. *Management Science*, 58(5), 892-912.
- [8] Greenberg, M. D., Pardo, B., Hariharan, K., & Gerber, E. (2013). Crowdfunding support tools : Predicting success & failure. In *CHI'13 Extended Abstracts on Human Factors in Computing Systems* (pp. 1815-1820).
- [9] Lukkarinen, A., Teich, J. E., Wallenius, H., & Wallenius, J. (2016). Success drivers of online equity crowdfunding campaigns. *Decision Support Systems*, 87, 26-38.
- [10] Cortellazzo, L., Bonesso, S., Gerli, F., & Batista-Foguet, J. M. (2020). Prototypicality and authenticity in crowdfunding : An fsQCA configurational approach. *International Journal of Research in Marketing*, 37(4), 721-738.
- [11] Meta Platforms, Inc. (2024). *React Documentation*. Consulté le 3 mai 2026. <https://react.dev/>
- [12] OpenJS Foundation. (2024). *Node.js Documentation*. Consulté le 3 mai 2026. <https://nodejs.org/>

- [13] MongoDB, Inc. (2024). *MongoDB Manual*. Consulté le 3 mai 2026. <https://www.mongodb.com/docs/>
- [14] n8n GmbH. (2024). *n8n Workflow Automation Documentation*. Consulté le 3 mai 2026. <https://n8n.io/>
- [15] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Consulté le 3 mai 2026. <https://scrumguides.org/>
- [16] Beck, K., Beedle, M., van Bennekum, A., et al. (2001). *Manifesto for Agile Software Development*. Consulté le 3 mai 2026. <https://agilemanifesto.org/>
- [17] OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. Consulté le 3 mai 2026. <https://owasp.org/Top10/>
- [18] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) - RFC 7519*. Internet Engineering Task Force (IETF).

Annexe

Format type d'une description textuelle de cas d'utilisation

Le tableau 4.3 reprend le modèle utilisé au chapitre 2 pour les fiches détaillées (section 2.5.2) et, le cas échéant, pour les autres cas listés aux tableaux 2.1 à 2.5.

Rubrique	Contenu attendu
Cas d'utilisation	Nom métier (verbe + objet, ex. <i>Soutenir un projet</i>).
Acteur(s)	Rôles métier déclencheurs et participants.
Précondition	État du système requis avant l'exécution.
Postcondition	État du système garanti après une exécution réussie.
Scénario nominal	1. Initiatives de l'acteur principal et sollicitations éventuelles du système (ordre chronologique).
	2. Enchaînements et validations côté application, services internes ou tiers.
	3. Conclusion nominale : résultat métier, persistance des données, effets annexes (notifications, etc.).
Scénarios alternatifs	1. Première variante (succès partiel, autre issue, branche optionnelle).
	2. Autres variantes le cas échéant (une ligne numérotée par variante dans cette colonne).
Exceptions	1. Erreurs de saisie, règles métier ou validations refusées.
	2. Défaillances techniques ou refus d'accès ; comportement observable (code retour, message).

TABLE 4.3 – Modèle de description textuelle d'un cas d'utilisation